

DS4023 (2025 Fall)
Machine Learning
Department of Computer Science
BNU-HKBU United International College



Chapter 2: Statistical Learning

Machine Learning $\approx$ Construct a mapping function (rule)

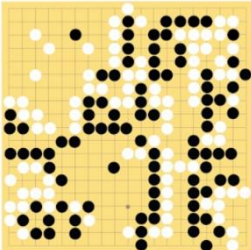
► Speech recognition

$f(\text{  }) = \text{“你好”}$

► Image recognition

$f(\text{  }) = \text{“9”}$

► 围棋

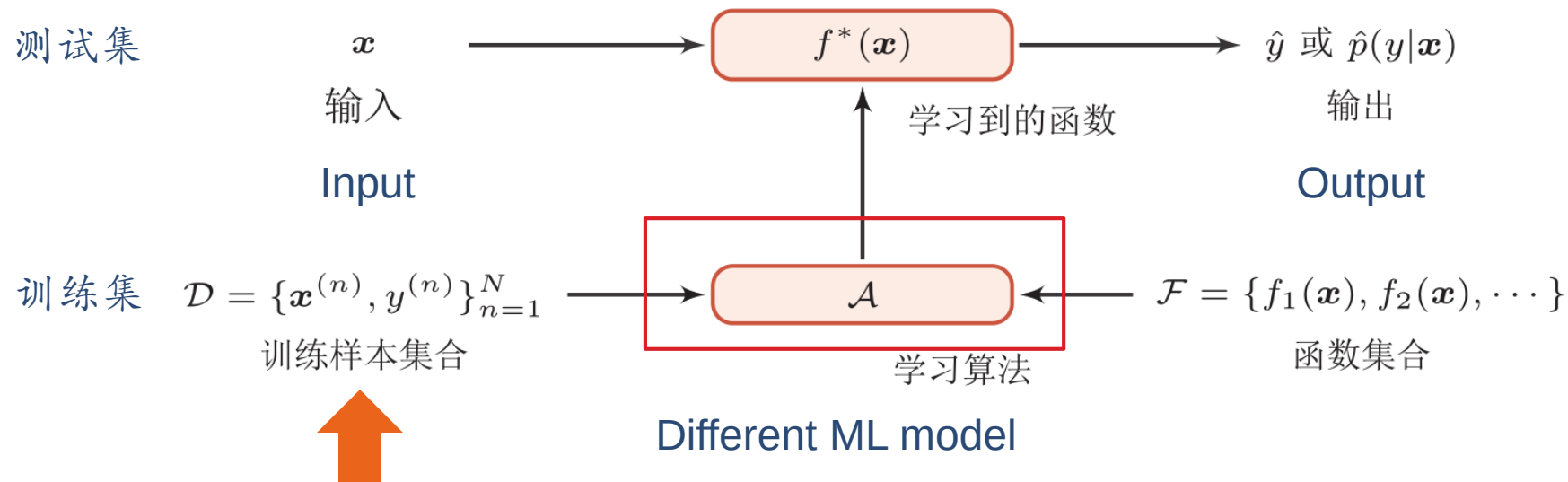
$f(\text{  }) = \text{“6-5”} \quad (\text{落子位置})$

► Machine Translation

$f(\text{“你好!”}) = \text{“Hello!”}$

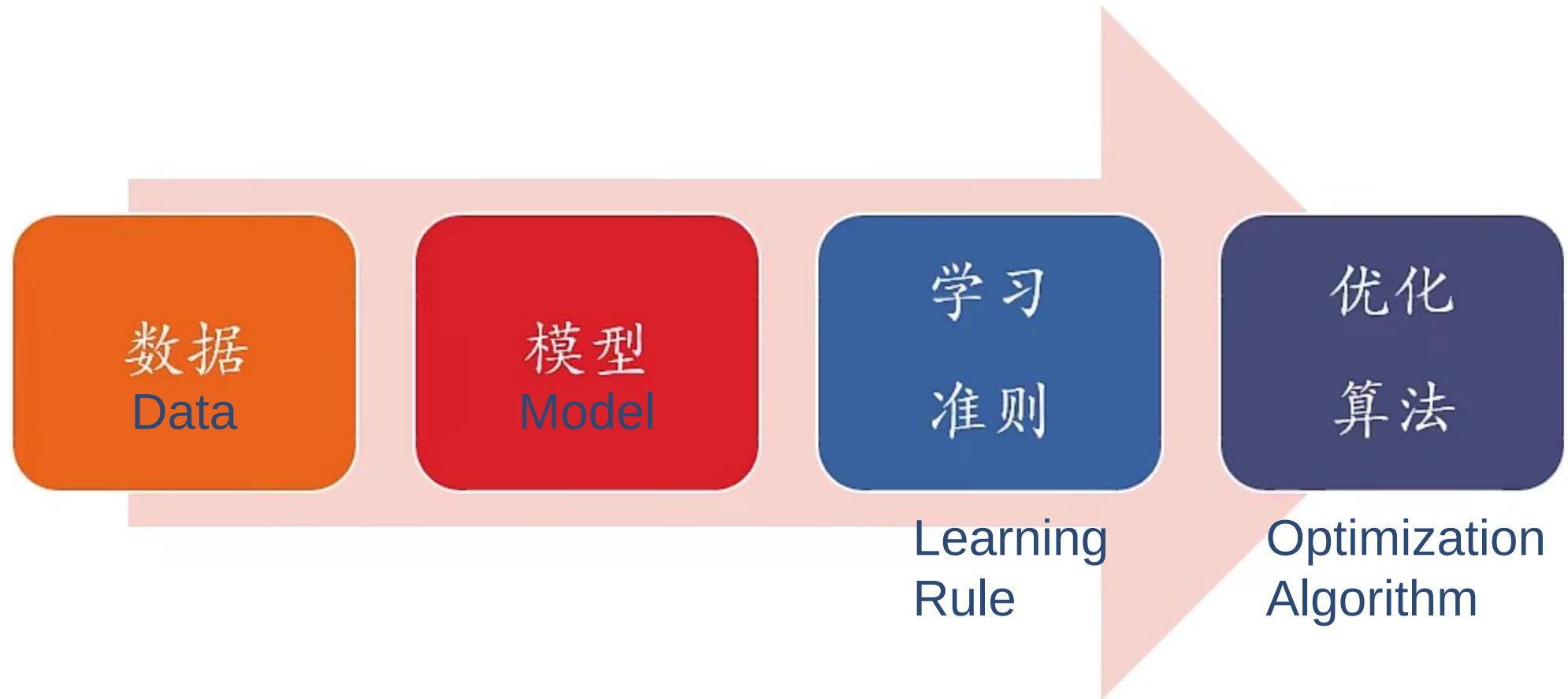
Definition of ML

- ▶ ML: through algorithms, machines can learn rules from a large number of data and make decisions on new sample
- ▶ Rules: decision (prediction) function



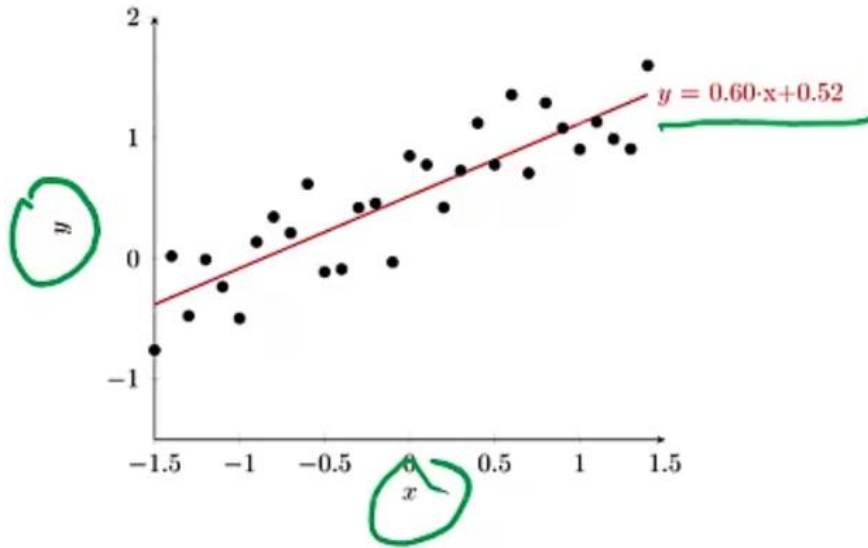
Independent and Identical distribution $p(\mathbf{x}, y)$

Four Elements in Machine Learning



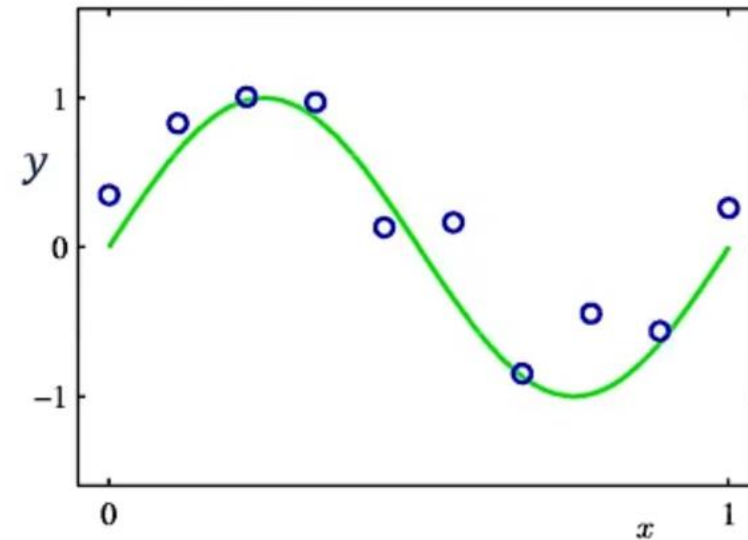
Model

► Taking regression as an example



Linear Model

$$f(\mathbf{x}; \theta) = \mathbf{w}^T \mathbf{x} + b$$



Nonlinear Model

$$f(\mathbf{x}; \theta) = \mathbf{w}^T \phi(\mathbf{x}) + b$$

If $\phi(x)$ is a learnable nonlinear basis function, $f(\mathbf{x}; \theta)$ is equivalent to a neural network.

Learning Rules

- ▶ A good model should be consistent with the real mapping function in all values

$$|f(\mathbf{x}, \theta^*) - y| \leq \epsilon, \forall (\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$$

- ▶ Loss function: it is a non-negative real function, used to quantify the difference between model's prediction and true label

- ▶ Take regression as an example:

- ▶ Quadratic loss function:

$$\mathcal{L}(y, f(\mathbf{x}; \theta)) = \frac{1}{2} (y - f(\mathbf{x}; \theta))^2$$

Learning Rules

► Expected Risk (期望风险)

$$\mathcal{R}(\theta) = \mathbb{E}_{(x,y) \sim p_r(x,y)} [\mathcal{L}(y, f(x; \theta))]$$

Real data distribution



Loss function



Learning Rules

- ▶ Expected risk is unknown, approximated by empirical risk

- ▶ Training data: $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$

- ▶ Empirical Risk:

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

- ▶ Empirical risk minimization

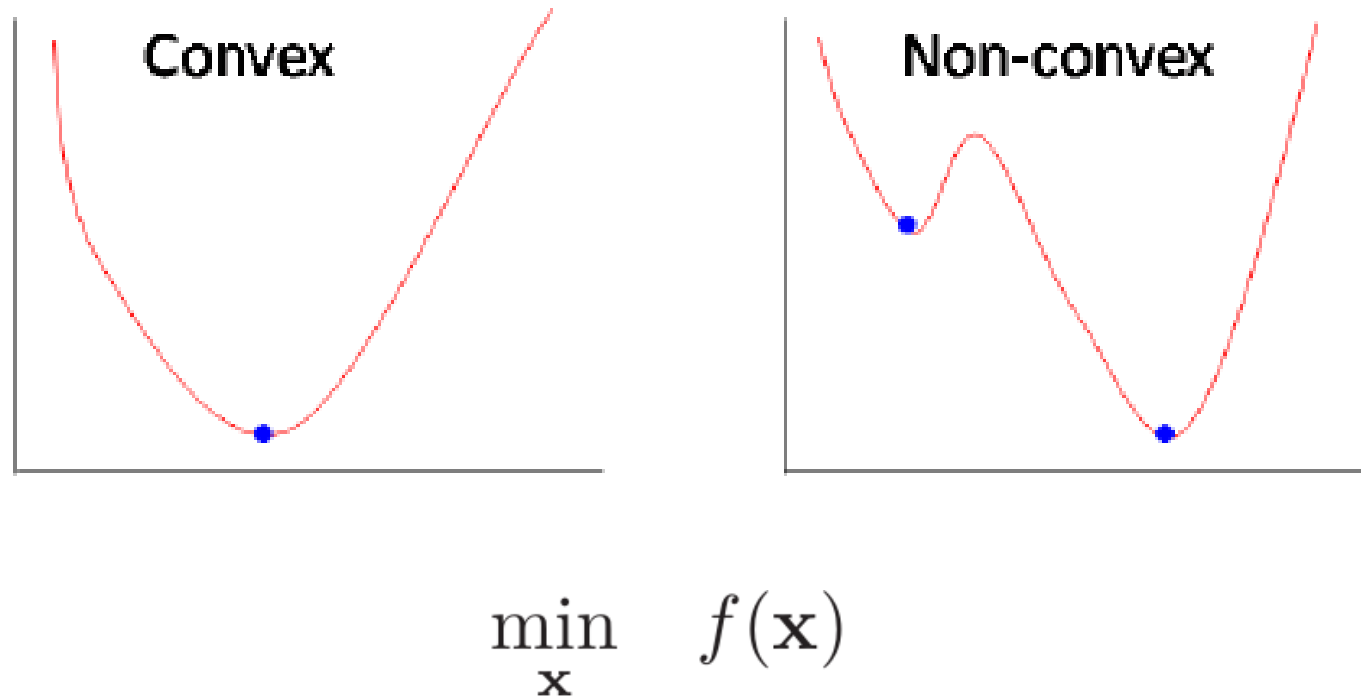
- ▶ After selecting the appropriate risk function, we look for a parameter θ^* to minimize the empirical risk function:

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

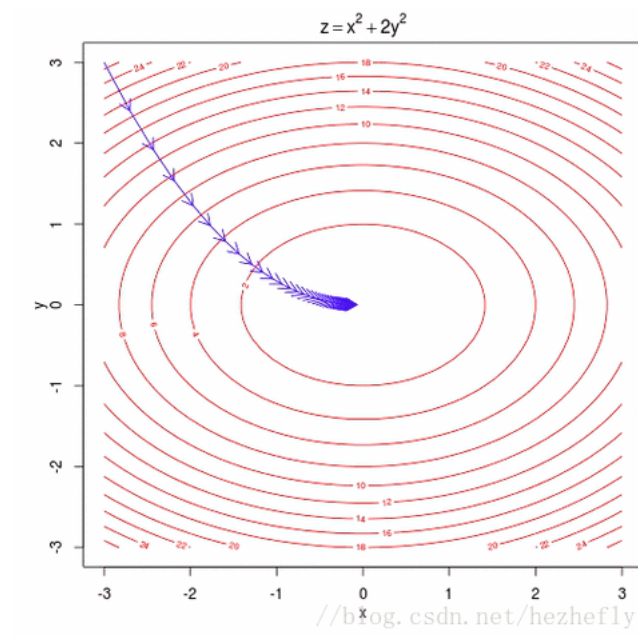
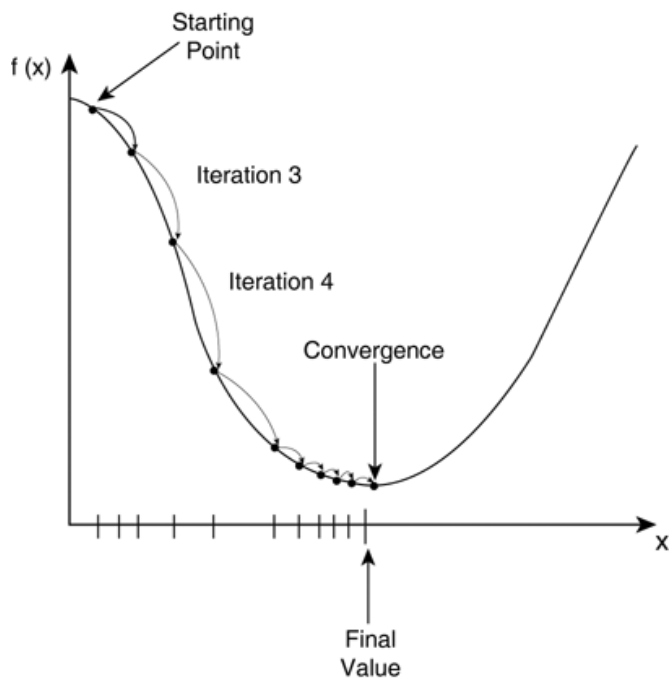
- ▶ ML problem is transformed into an optimization problem

Optimization Problems

- ML problems are transformed into optimization problems



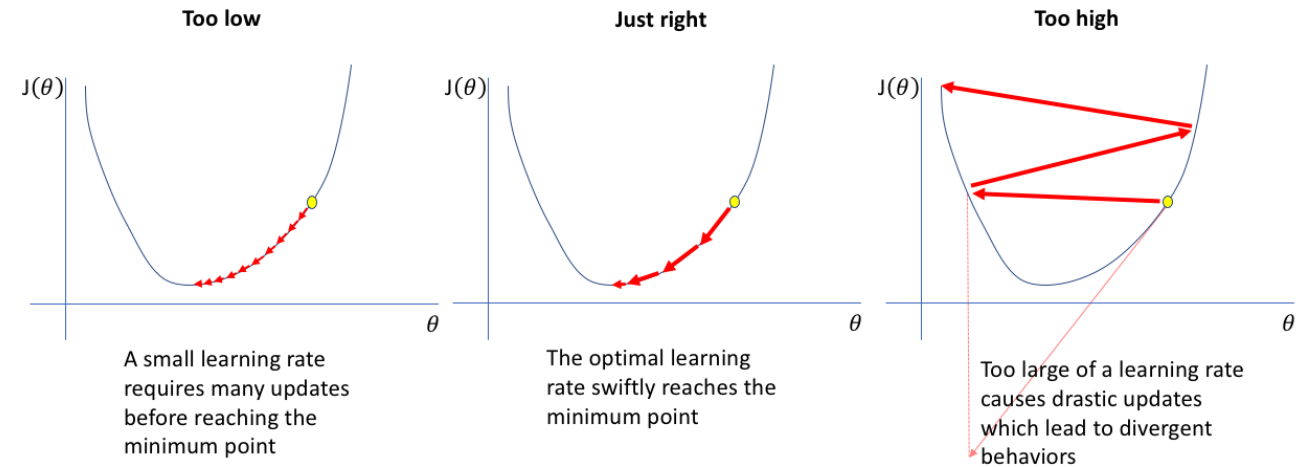
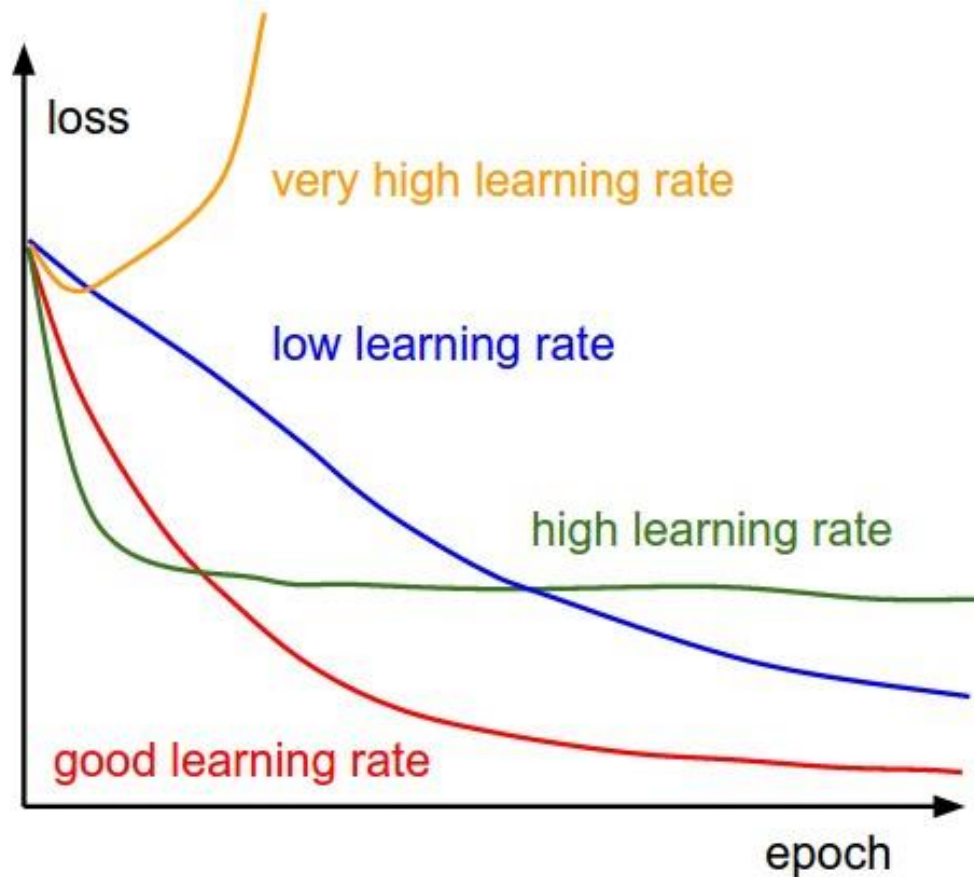
Gradient Descent (梯度下降法)



$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}_{\mathcal{D}}(\theta)}{\partial \theta} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{n=1}^N \frac{\partial \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta))}{\partial \theta},\end{aligned}$$

The search step size α is also called **Learning Rate**

Learning rate is a very important Hyperparameter!



Stochastic Gradient Descent (SGD, 随机梯度下降法)

► SGD: sampling one samples in each iteration

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{L}(y^n, f(\mathbf{x}^n; \theta))}{\partial \theta}, n = 1 \cdots N$$

- After enough iterations, SGD can also converge to the local optimal solution.
- Good: Overhead of each calculation is small, support online learning
- Bad: Can't make full use of the computer's parallel computing ability

► Mini-Batch SGD

- Randomly select a small number of training samples to calculate gradient and update parameters
- It can not only take into account the advantages of SGD method, but also improve training efficiency
- Batch size is another important hyperparameter

Stochastic Gradient Descent

算法 2.1: 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

1 随机初始化 θ ;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机重排序;

4 **for** $n = 1 \cdots N$ **do**

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

 // 更新参数

6 $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta}$;

7 **end**

8 **until** 模型 $f(\mathbf{x}; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: θ

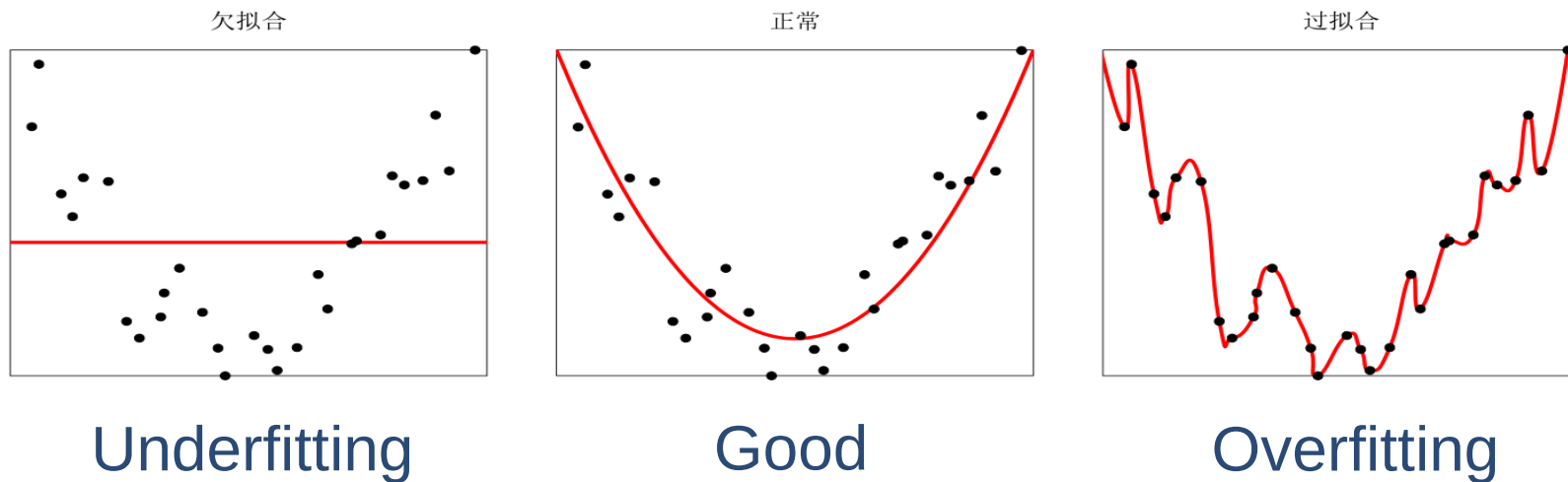


Why?

Machine Learning = Optimization?

ML = Optimization?

NO!



Overfitting: the principle of **empirical risk minimization** can easily lead to a low error rate in the training set, but a high error rate in the unknown data.

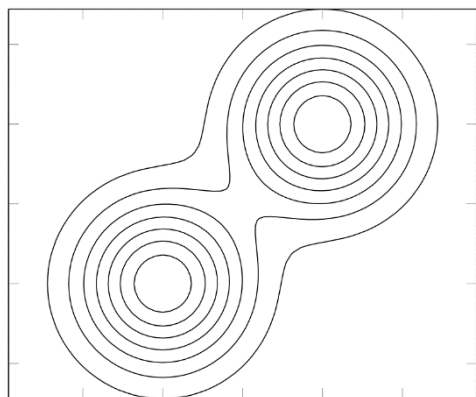
Overfitting problems are often caused by less training data and noise.

Generalization Error (泛化误差)

Expected Risk

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y)} [\mathcal{L}(f(\mathbf{x}), y)],$$

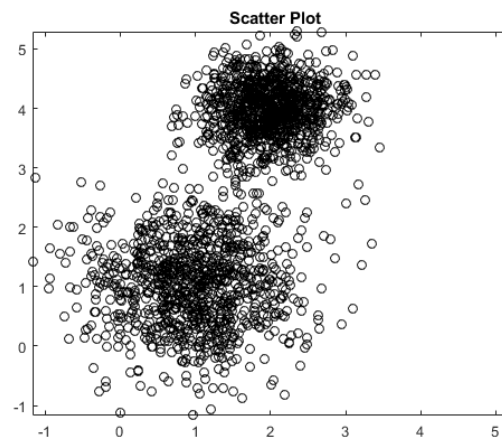
真实分布 p_r



$\neq$

Empirical Risk

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$



$$\mathcal{G}_{\mathcal{D}}(f) = \mathcal{R}(f) - \mathcal{R}_{\mathcal{D}}^{emp}(f)$$

Generalization error

How to reduce generalization errors?

Optimization: minimize empirical Risk

Regularization (正则化): Reduce model complexity



Regularization (正则化)

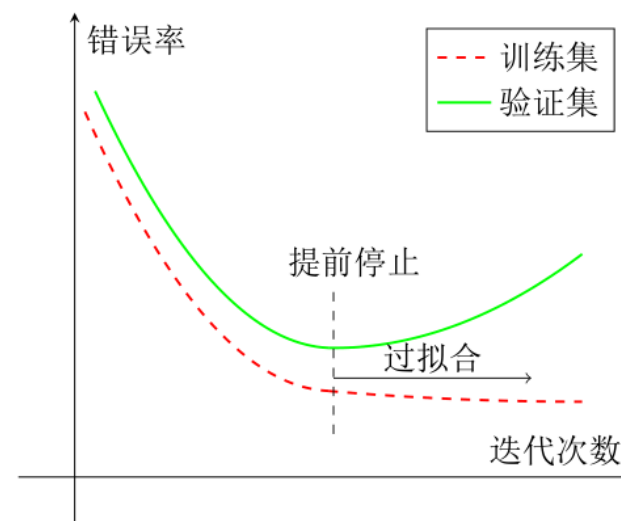
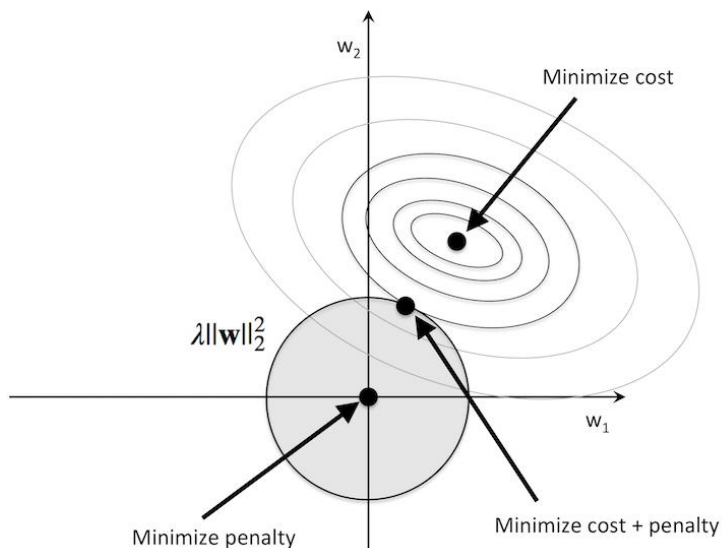
All damage optimization methods are regularization.

Add optimization constraints

L1/L2 constraints, Data enhancement

Interfere optimization process

Weight attenuation, SGD, Early Stop





2.1 Linear Regression

Regression (回归)

► **Training Set:** $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$

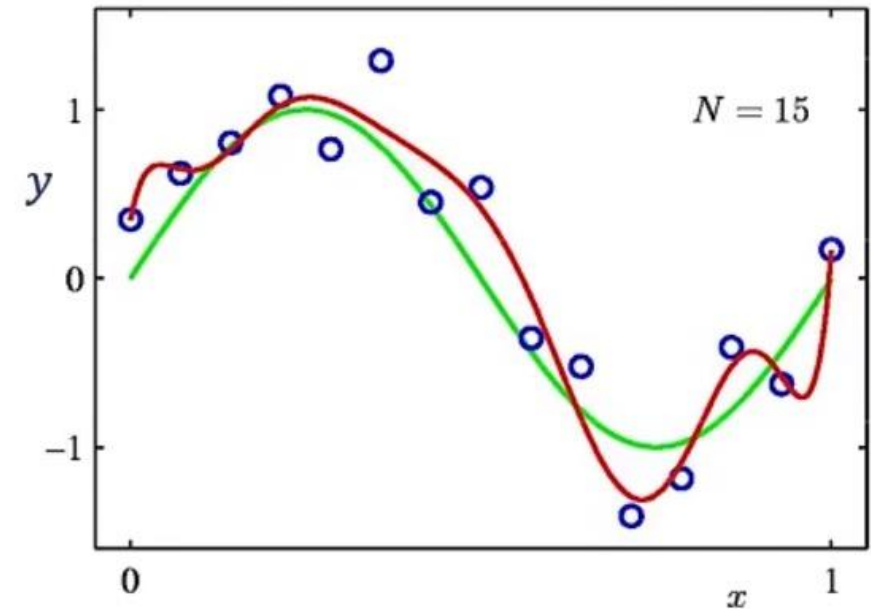
► **Regression problem:**

► $\mathbf{x}^{(n)} \in \mathbb{R}^D$

► $y^{(n)} \in \mathbb{R}$ is continuous variable

► Find the mapping relationship between $\mathbf{x}$ and y

$$y = f(\mathbf{x}; \boldsymbol{\theta})$$

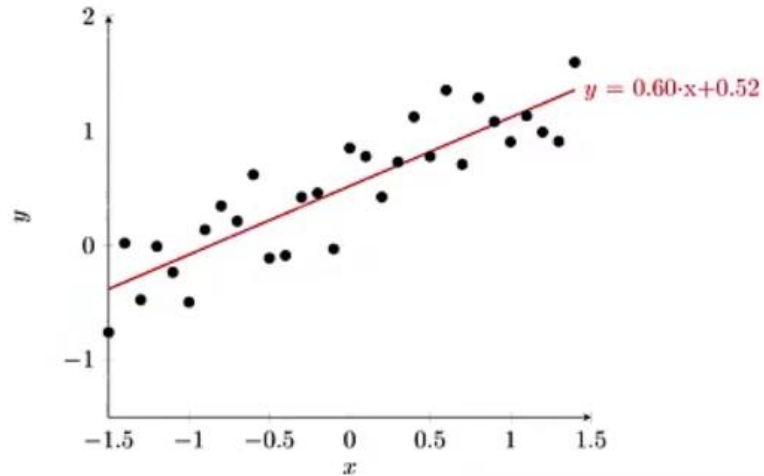


Linear Regression (线性回归)

- ▶ **Model:** $f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$
- ▶ For convenience, augment weight vector and feature vector

$$\hat{\mathbf{x}} = \mathbf{x} \oplus 1 \triangleq \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_D \\ 1 \end{bmatrix}$$

$$\hat{\mathbf{w}} = \mathbf{w} \oplus b \triangleq \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \begin{bmatrix} w_1 \\ \vdots \\ w_D \\ b \end{bmatrix}$$



$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

Empirical Risk Minimization (ERM)

► Empirical risk in training set $\mathcal{D}$

► $\mathbf{y} = [y^{(1)}, \dots, y^{(N)}] \in \mathbb{R}^N$ is a column vector composed of true labels

$$\begin{aligned}\mathcal{R}(\mathbf{w}) &= \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \mathbf{w})) \\ &= \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \mathbf{x}^{(n)} \right)^2 \\ &= \frac{1}{2} \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2,\end{aligned} \quad \mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(N)} \\ \vdots & \vdots & \ddots & \vdots \\ x_D^{(1)} & x_D^{(2)} & \dots & x_D^{(N)} \\ 1 & 1 & \dots & 1 \end{bmatrix}.$$

► To minimize the Empirical risk

$$\text{令 } \frac{\partial}{\partial \mathbf{w}} \mathcal{R}(\mathbf{w}) = 0$$

Matrix Calculus

► Partial derivative of scalar w.r.t. vector $\frac{\partial y}{\partial \mathbf{x}} = [\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_M}]^\top$

► P.d. of vector w.r.t. vector

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_N}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_M} & \dots & \frac{\partial y_N}{\partial x_M} \end{bmatrix}$$

► Vector function:

$$\begin{aligned} \frac{\partial \mathbf{x}}{\partial \mathbf{x}} &= \mathbf{I}, \\ \frac{\partial \mathbf{A}\mathbf{x}}{\partial \mathbf{x}} &= \mathbf{A}^\top, \\ \frac{\partial \mathbf{x}^\top \mathbf{A}}{\partial \mathbf{x}} &= \mathbf{A} \end{aligned}$$

经验风险最小化

$$\begin{aligned}\frac{\partial \mathcal{R}(\mathbf{w})}{\partial \mathbf{w}} &= \frac{1}{2} \frac{\partial \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2}{\partial \mathbf{w}} \\ &= -\mathbf{X}(\mathbf{y} - \mathbf{X}^\top \mathbf{w}),\end{aligned}$$

$$\mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$$

► **Set** $\frac{\partial}{\partial \mathbf{w}} \mathcal{R}(\mathbf{w}) = 0$, **get optimal para.**

$$= \left(\sum_{n=1}^N \mathbf{x}^{(n)} (\mathbf{x}^{(n)})^\top \right)^{-1} \left(\sum_{n=1}^N \mathbf{x}^{(n)} y^{(n)} \right).$$

► **Least square method**

- $\mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{(D+1) \times (D+1)}$ must have inverse matrix (full rank)
- $\mathbf{X}$'s row vectors are linearly independent: each feature is linearly independent of other features
- Common irreversibility: sample number $N <$ feature number $D+1$

Structure Risk Minimization (结构风险最小化, SRM)

▶ When collinearity between features, XX^T irreversable

- ▶ A small disturbance on the dataset will lead to a great change in $(XX^T)^{-1}$, which will make the calculation unstable.
- ▶ Now, can get multiple solutions w^* . Which one is better?

▶ Structure Risk

$$\mathcal{R}(w) = \frac{1}{2} \|y - X^T w\|^2 + \frac{1}{2} \lambda \|w\|^2,$$

▶ Minimize to get $w^* = (XX^T + \lambda I)^{-1} Xy$,

▶ This is called Ridge Regression (岭回归)

Variant: Log-linear regression

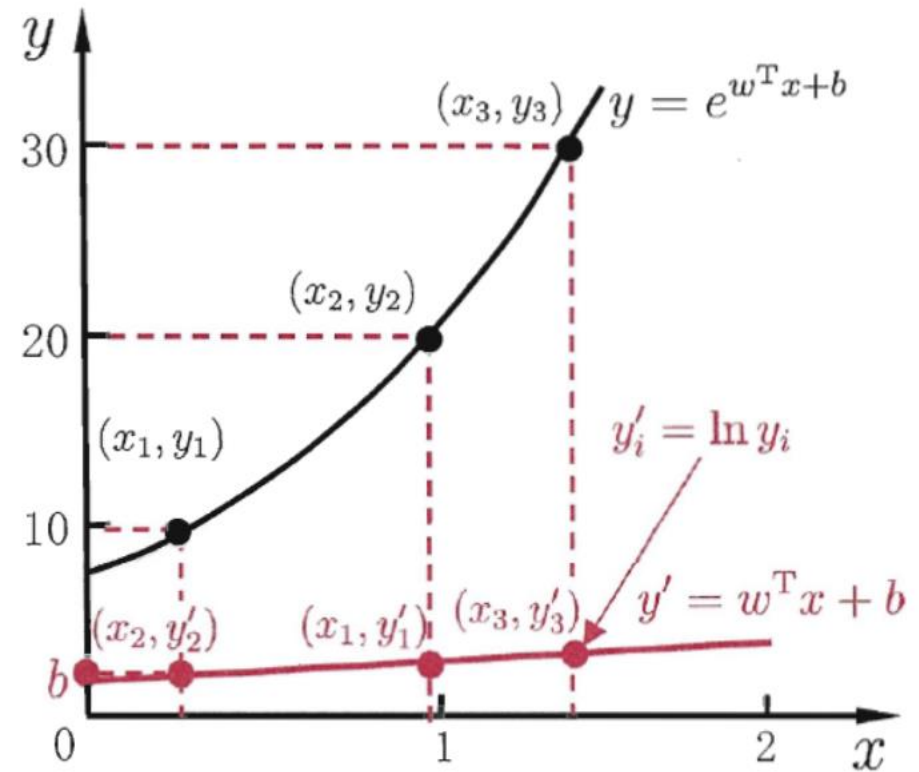
► $\ln y = \mathbf{w}^T \mathbf{x} + b$

► It is linear regression, but find the nonlinear mapping from input space to output space:

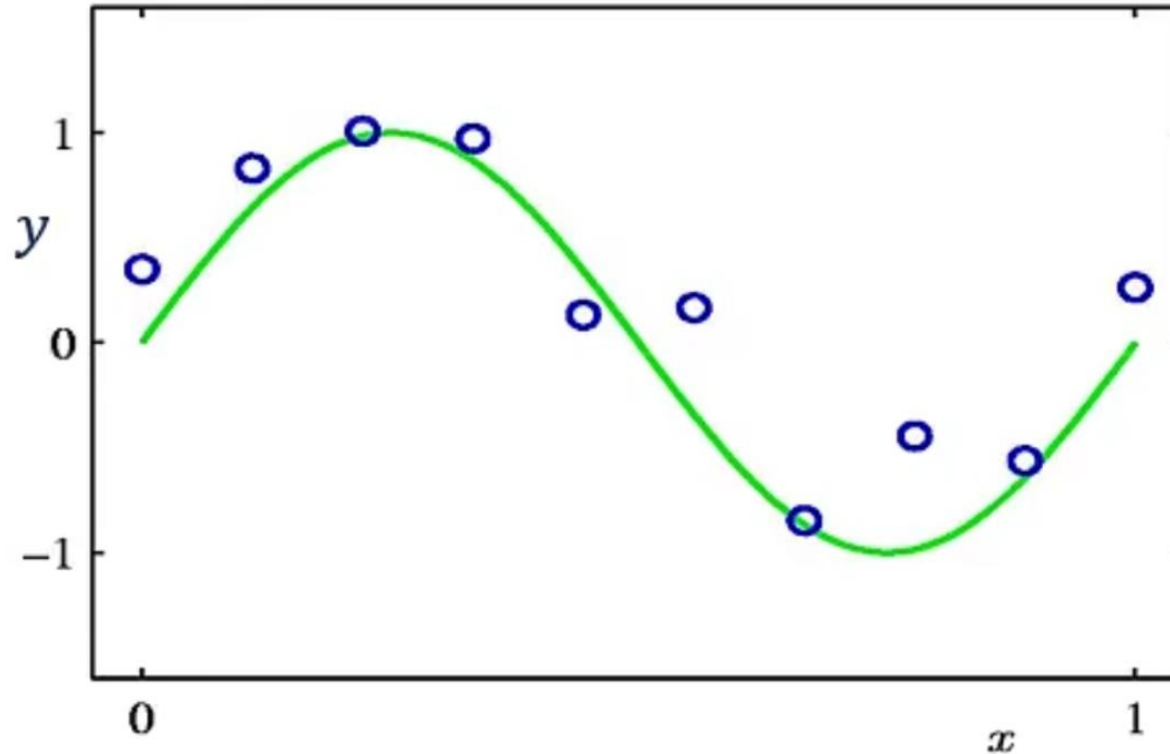
$$y = e^{\mathbf{w}^T \mathbf{x} + b}$$

► More generally, consider differentiable function $g(\cdot)$: link function

► We have $y = g^{-1}(\mathbf{w}^T \mathbf{x} + b)$, called “generalized linear model



Curve Fitting (曲线拟合)

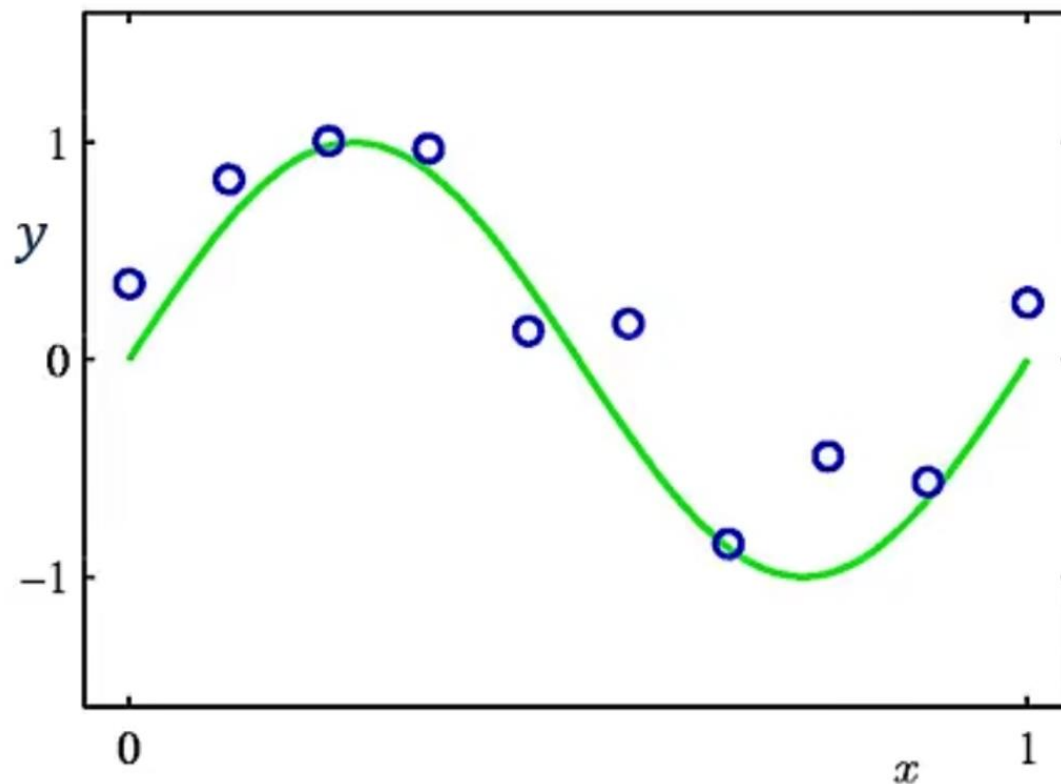


Given the Training Set: $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, where $\mathbf{x}^{(n)} \in \mathbb{R}^D$ and $y^{(n)} \in \mathbb{R}$ are continuous variables, to find the mapping relationship between $\mathbf{x}$ and y : $y = f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \phi(\mathbf{x}) + b$

Polynomial Curve Fitting (多项式曲线拟合)

► Model definition:

$$f(\boldsymbol{x}; \boldsymbol{w}) = w_0 + w_1x + w_2x^2 + \cdots + w_Mx^M$$



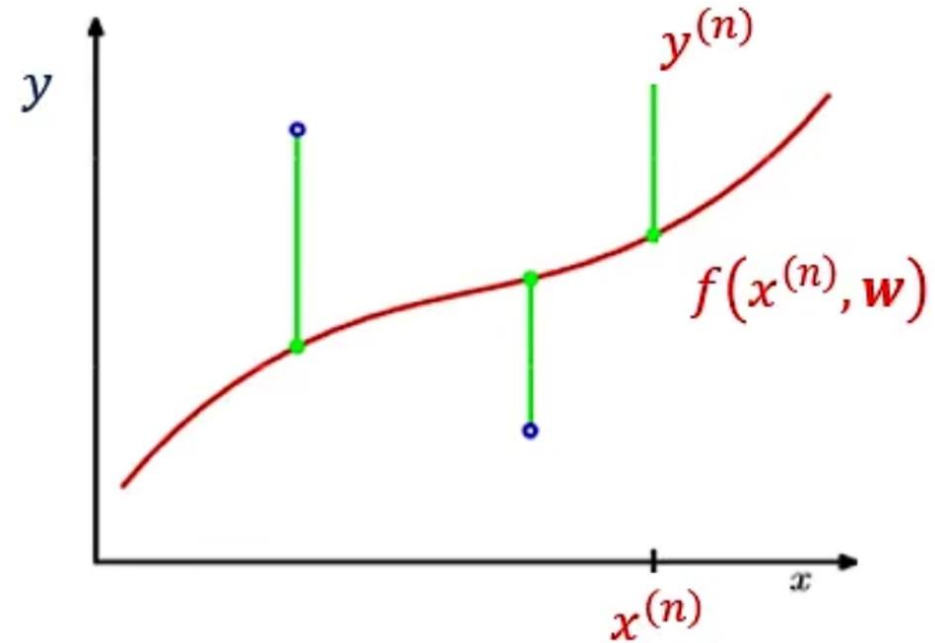
Loss function

► Sum-of-Squares (平方误差)

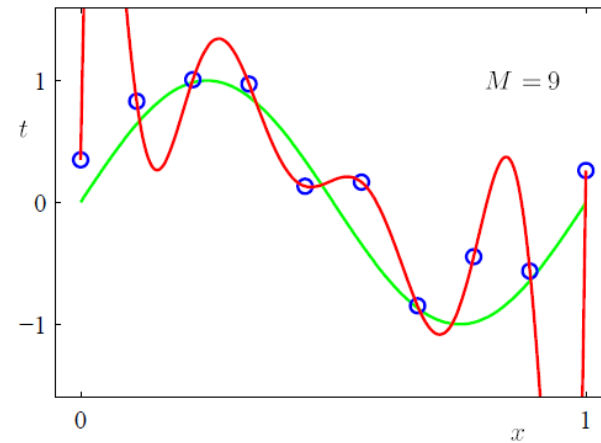
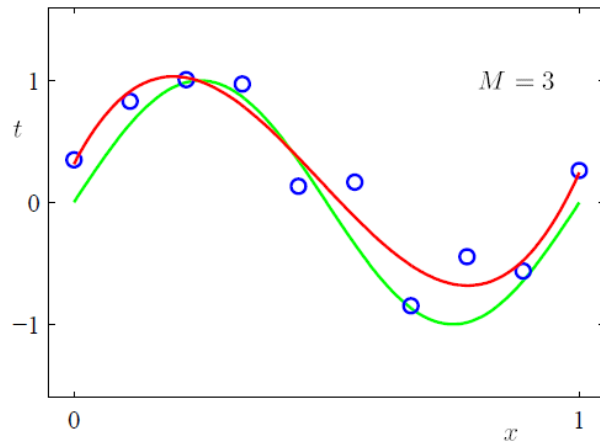
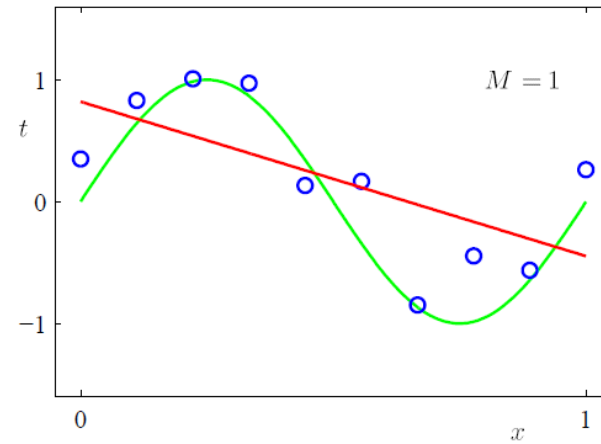
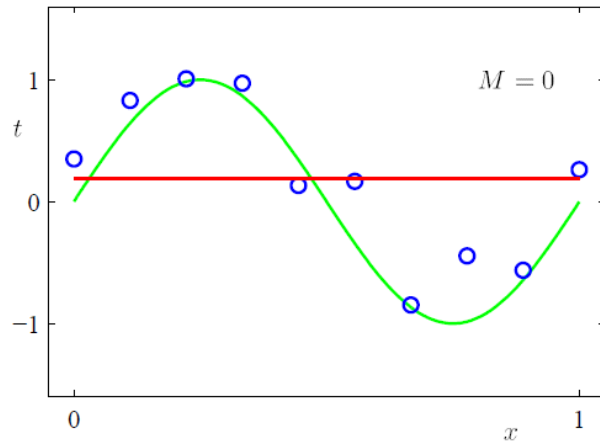
$$R(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^T \phi(x^{(n)}) \right)^2$$

► Minimize empirical risk

- Solution process is similar to linear regression
- M is a hyperparameter



How to Choose Polynomial Degree?

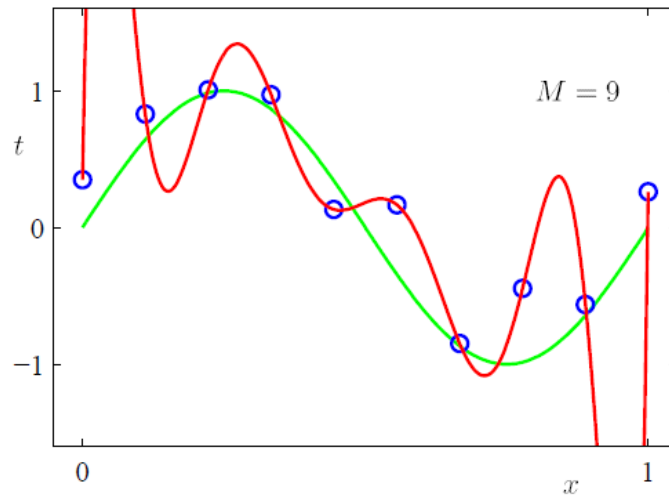


模型选择问题

A model selection problem

$M = 9 \rightarrow E(w) = 0$: This is overfitting

How to Choose Polynomial Degree?



	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

As order of polynomial M increases, so do coefficient magnitudes!
Punish large coefficients.

Control Overfitting: Regularization

- Punish large coefficients:

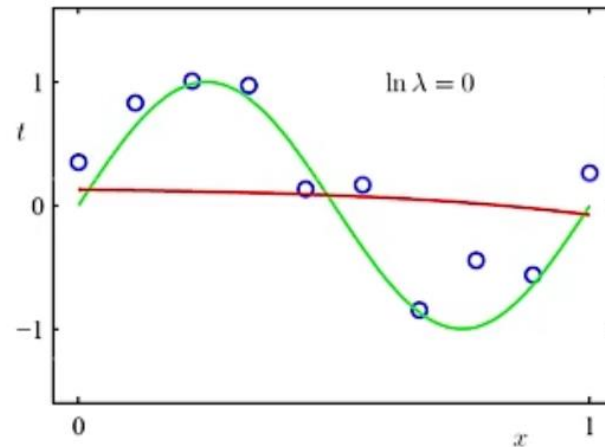
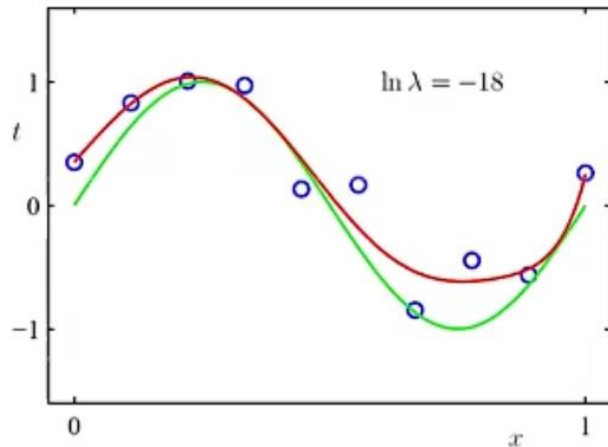
$$\hat{\mathcal{R}}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \phi(\mathbf{x}^{(n)}) \right)^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}$$

Regularization term 正则化项

- λ is regularization coefficient (正则化系数)

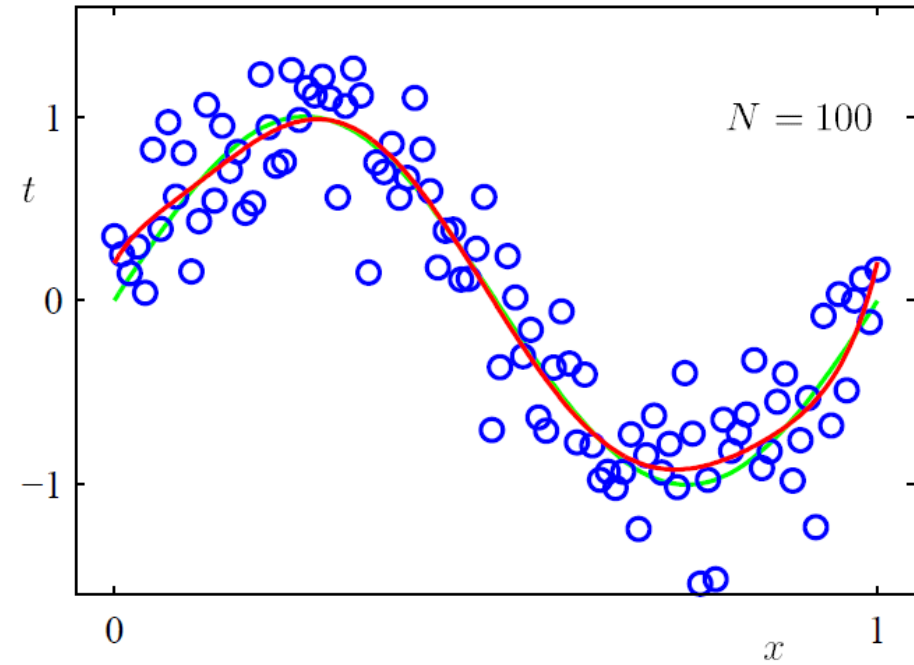
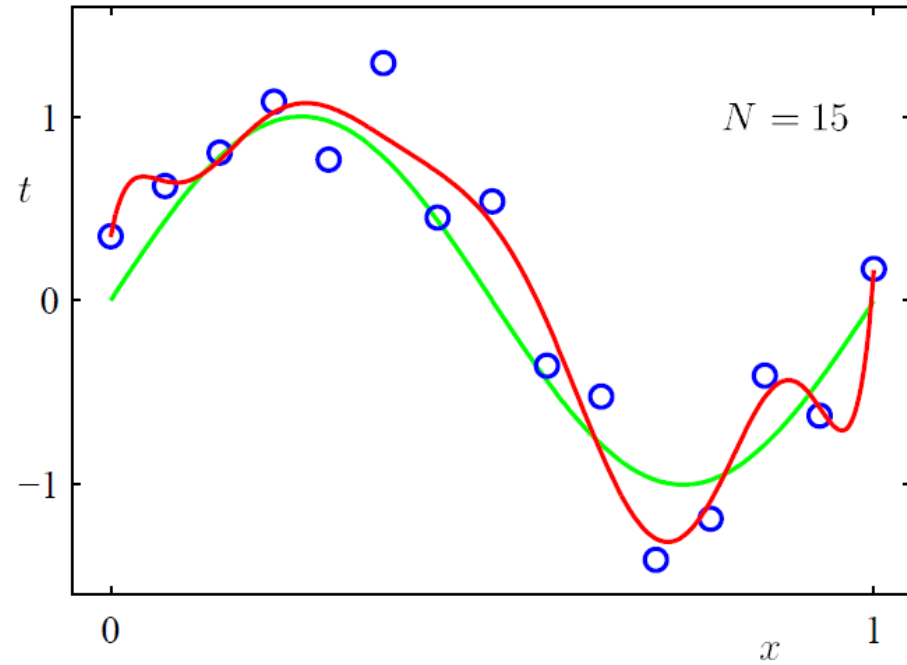
Control Overfitting: Regularization

$$\hat{\mathcal{R}}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \phi(\mathbf{x}^{(n)}) \right)^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}$$



	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Control Overfitting: Increase training samples





2.2 Probability Distribution

Some Basic Concepts about Probability

► Probability

- Probability of a random event is a real number between 0 and 1

► Random variable (r.v.)

- Real single-valued functions of various results of random tests
- For example, if a dice is randomly thrown, the number of points obtained can be regarded as a r.v X , and its value is $\{1,2,3,4,5,6\}$

► Probability Distribution

- Probability that a r.v. X takes every possible value.

$$P(X = x_i) = p(x_i), \quad \forall i \in \{1, \dots, n\}.$$

- Satisfying:

$$\sum_{i=1}^n p(x_i) = 1, \quad p(x_i) \geq 0, \quad \forall i \in \{1, \dots, n\}.$$

Random Variable

► Discrete random variable

► Bernoulli Distribution

- In a test, event A happens with probability μ , does not happen with probability $1 - \mu$
- If using r.v. X to indicate the number of occurrences of event A, then X can be 0 or 1, its distribution is

$$p(x) = \mu^x (1 - \mu)^{1-x}$$

Random Variable

► Discrete random variable

► Binomial Distribution

- In the n times Bernoulli distribution, if r.v. X represents the number of occurrences of event A , the value of X is $\{0, \dots, n\}$, and its corresponding distribution

$$p(X = k) = \binom{n}{k} \mu^k (1 - \mu)^{n-k}, k = 1, \dots, n$$

- Binomial coefficient represents the total number of combinations of elements taken out of elements regardless of their order.

Random Variable

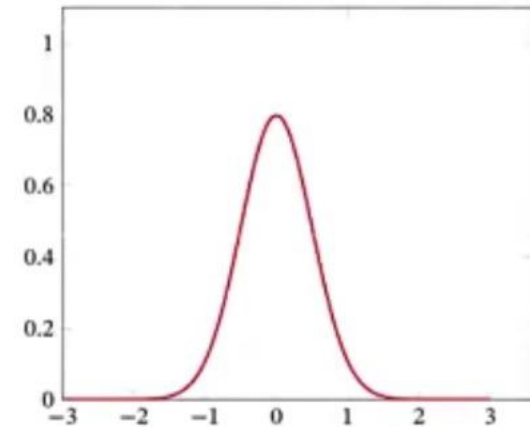
► Continuous random variable

- Probability distribution can be described by Probability Density function (PDF) $f(x)$:

$$\int_{-\infty}^{+\infty} f(x) dx = 1$$

- Gaussian Distribution:

$$X \sim \mathcal{N}(\mu, \sigma^2) \quad p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$



Random Variable

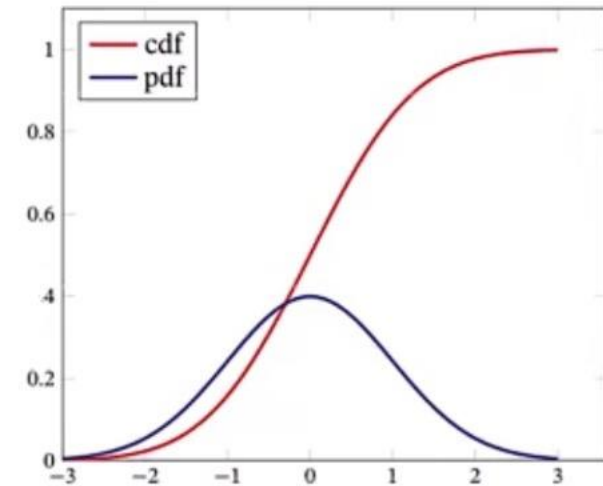
► Continuous random variable

- Cumulative Distribution Function (CDF) is the probability of the value of r.v. X less than or equal to x :

$$F(x) = P(X \leq x)$$

- For continuous r.v., we have

$$F(x) = \int_{-\infty}^x f(t) dt$$



Random Vector

► A vector consisting of a set of random variables

► Multivariate distribution

► Discrete r.v.: Joint Probability Distribution

$$P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = p(x_1, x_2, \dots, x_n)$$

► Continuous r.v.: Joint probability density $f(x_1, x_2, \dots, x_n)$

► Joint Distribution Function:

$$\begin{aligned} F(x_1, x_2, \dots, x_n) &= P(X_1 \leq x_1, X_2 \leq x_2, \dots, X_n \leq x_n) \\ &= \int_{-\infty}^{x_1} \int_{-\infty}^{x_2} \dots \int_{-\infty}^{x_n} f(t_1, t_2, \dots, t_n) dt_1 dt_2 \dots dt_n \end{aligned}$$

Marginal Distribution

► Marginal probability mass function

- Given a known joint distribution of two discrete r.v. X and Y , the marginal distribution of either variable – X for example – is the probability distribution of X when the values of Y are not taken into consideration.

$$p_X(x_i) = \sum_j p(x_i, y_j) \text{ or } p_Y(y_j) = \sum_i p(x_i, y_j)$$

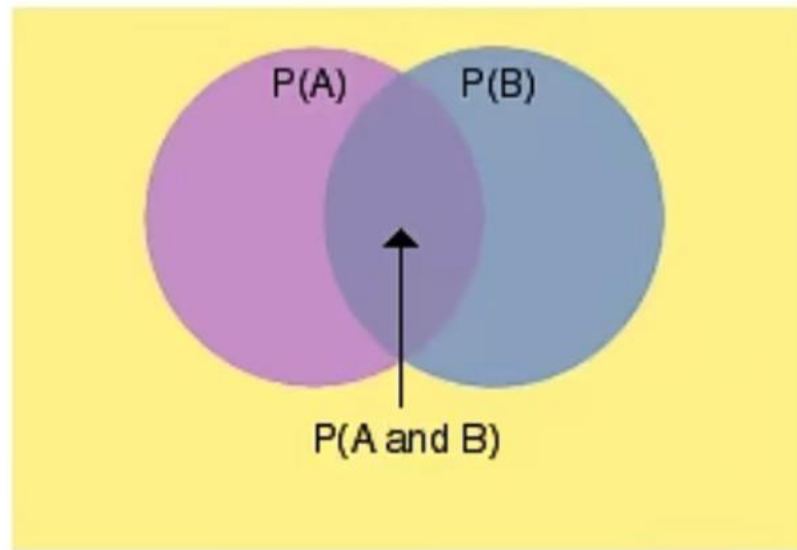
► Marginal probability density function

$$f_X(x) = \int_{-\infty}^{+\infty} f(x, y) dy \text{ or } f_Y(y) = \int_{-\infty}^{+\infty} f(x, y) dx$$

Conditional Probability

- For a discrete random vector (X, Y) , when $X = x$ is known, the conditional probability of r.v. $Y = y$ is

$$p(y|x) = P(Y = y|X = x) = \frac{p(x, y)}{p(x)}$$



An example of conditional probability

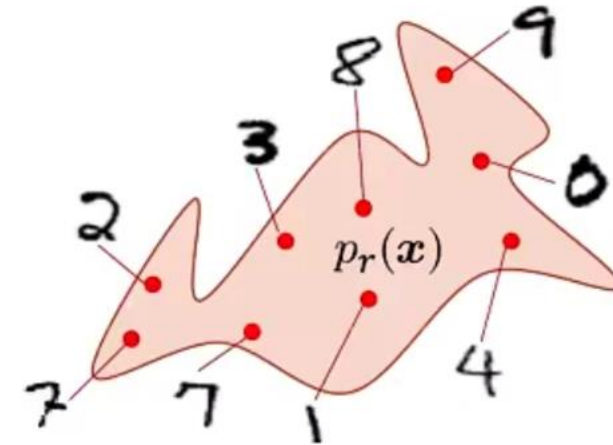
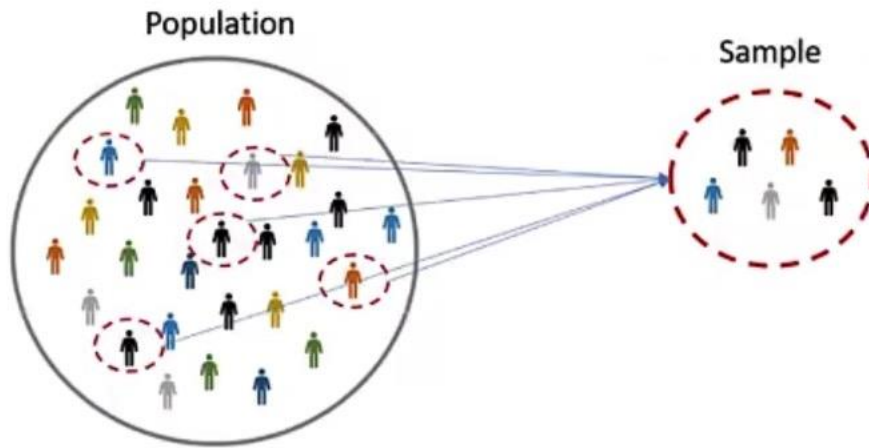
		Marginal Probability	
性别\行业	计算机	教育	
男	0.4	0.1	0.5
女	0.1	0.4	0.5
Marginal Probability	0.5	0.5	

$$p(\text{男}|\text{计算机})=$$

Sampling

- Sampling: given a probability distribution $p(x)$, generate samples that meet the conditions

$$x^{(1)}, x^{(2)}, \dots, x^{(N)} \sim p(x)$$



How to conduct random sampling?

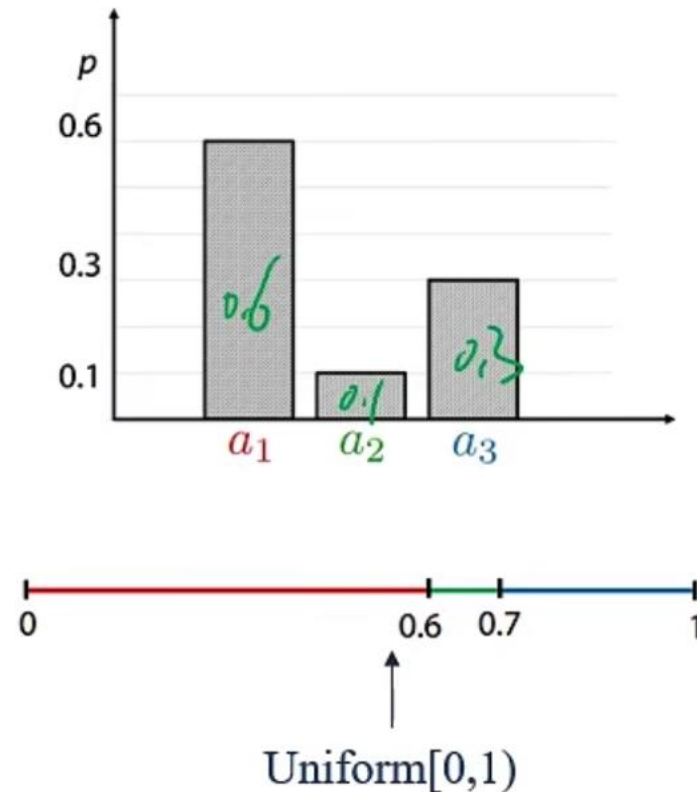
► Direct sampling

► Uniform distribution: Uniform[0,1)

线性同余发生器 (Linear congruential generator)

```
static unsigned long int next = 1; // 种子
int rand(void) // 生成伪随机数
{
    next = next * 1103515245 + 12345;
    return (unsigned int) (next / 65536) % 32768;
}
void srand(unsigned int seed) // 修改种子
{
    next = seed;
}
```

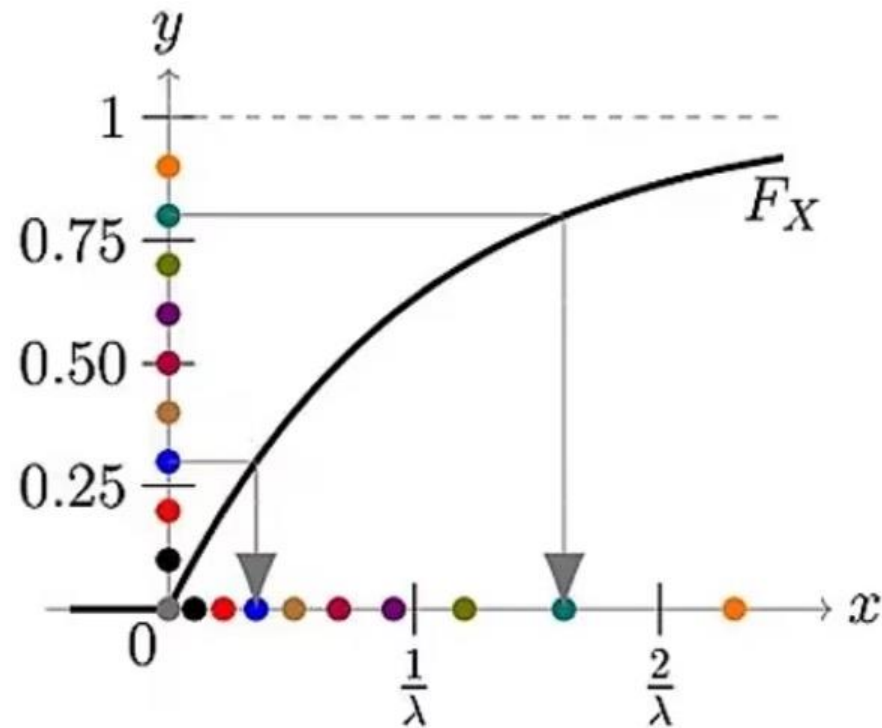
► Discrete distribution



How to conduct random sampling?

► Other continuous distribution

► For example, Inverse Transform Sampling (逆变换采样)



Expectation

- ▶ Expectation: the average of random variable
- ▶ For discrete r.v. X :

$$\mathbb{E}[X] = \sum_{n=1}^N x_n p(x_n)$$

- ▶ For continuous r.v. X :

$$\mathbb{E}[X] = \int_{-\infty}^{+\infty} x f(x) d(x)$$

Law of Large Numbers

- ▶ When the number of samples is large, the sample mean and the real mean (expectation) are fully close.
- ▶ Given N independently and identically distributed (I.I.D.) samples

$$x^{(1)}, x^{(2)}, \dots, x^{(N)} \sim p(x)$$

- ▶ Its mean value converges to the expected value:

$$\bar{x}_N = \frac{1}{N} (x^{(1)} + x^{(2)} + \dots + x^{(N)}) \rightarrow \mathbb{E}[X] \text{ for } N \rightarrow \infty$$



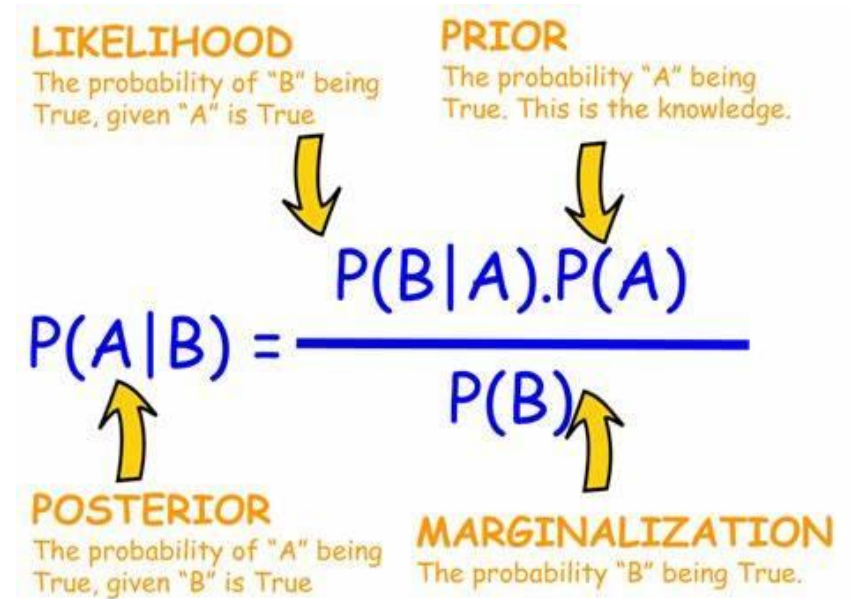
2.3 Naïve Bayes Classification

Bayesian classification

- ▶ **Probabilistic view:** Supervised learning can naturally be studied from a probabilistic point of view.
- ▶ Let $A = \{A_1, A_2, \dots\}$ be attribute set with discrete values. Class is C .
- ▶ Given a test example d with observed attribute values $a_1, a_2, \dots$.
- ▶ Classification is basically to compute the following posteriori probability. The prediction is the class c_j such that
$$\Pr(C = c_j | A_1 = a_1, \dots, A_{|A|} = a_{|A|})$$
is maximized.

Applying Bayes' rule

$$\begin{aligned} & \Pr(C = c_j | A_1 = a_1, \dots, A_{|A|} = a_{|A|}) \\ &= \frac{\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|} | C = c_j) \Pr(C = c_j)}{\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|})} \\ &= \frac{\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|} | C = c_j) \Pr(C = c_j)}{\sum_{r=1}^{|C|} \Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|} | C = c_r) \Pr(C = c_r)} \end{aligned}$$



- $\Pr(C=c_j)$ is the class *prior* probability: easy to estimate from the training data.

Computing probabilities

- ▶ The denominator $\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|})$ is irrelevant for decision making since it is the same for every class.
- ▶ We only need $\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|} | C = c_j)$, which can be written as $\Pr(A_1 = a_1 | A_2 = a_2, \dots, A_{|A|} = a_{|A|}, C = c_j) \cdot \Pr(A_2 = a_2, \dots, A_{|A|} = a_{|A|} | C = c_j)$
- ▶ Recursively, the second factor above can be written in the same way, and so on.
- ▶ Now an assumption is needed.

Conditional Independence Assumption

- ▶ All attributes are conditionally independent given the class

$$C = c_j$$

- ▶ Formally, we assume

$$\begin{aligned} & \Pr(A_1 = a_1 | A_2 = a_2, \dots, A_{|A|} = a_{|A|}, C = c_j) \\ &= \Pr(A_1 = a_1 | C = c_j) \end{aligned}$$

and so on for A_2 through $A_{|A|}$.

- ▶ Thus, we have

$$\Pr(A_1 = a_1, \dots, A_{|A|} = a_{|A|} | C = c_j) = \prod_{i=1}^{|A|} \Pr(A_i = a_i | C = c_j)$$

Final Naïve Bayesian Classifier

$$\begin{aligned} & \Pr(C = c_j | A_1 = a_1, \dots, A_{|A|} = a_{|A|}) \\ &= \frac{\Pr(C = c_j) \prod_{i=1}^{|A|} \Pr(A_i = a_i | C = c_j)}{\sum_{r=1}^{|C|} \Pr(C = c_r) \prod_{i=1}^{|A|} \Pr(A_i = a_i | C = c_r)} \end{aligned}$$

We are done!

How do we estimate $P(A_i = a_i | C = c_j)$? Easy!.

Classify a test instance

- ▶ If we only need a decision on the most probable class for the test instance, we only need the numerator as its denominator is the same for every class.
- ▶ Thus, given a test example, we compute the following to decide the most probable class for the test instance

$$c = \operatorname{argmax}_{c_j} \Pr(c_j) \prod_{i=1}^{|A|} \Pr(A_i = a_i | C = c_j)$$

An Example

► Compute all probabilities required for classification

A	B	C
m	b	t
m	s	t
g	q	t
h	s	t
g	q	t
g	q	f
g	s	f
h	b	f
h	q	f
m	b	f

$$\Pr(C = t) = 1/2,$$

$$\Pr(C = f) = 1/2$$

$$\Pr(A=m \mid C=t) = 2/5$$

$$\Pr(A=g \mid C=t) = 2/5$$

$$\Pr(A=h \mid C=t) = 1/5$$

$$\Pr(A=m \mid C=f) = 1/5$$

$$\Pr(A=g \mid C=f) = 2/5$$

$$\Pr(A=h \mid C=f) = 2/5$$

$$\Pr(B=b \mid C=t) = 1/5$$

$$\Pr(B=s \mid C=t) = 2/5$$

$$\Pr(B=q \mid C=t) = 2/5$$

$$\Pr(B=b \mid C=f) = 2/5$$

$$\Pr(B=s \mid C=f) = 1/5$$

$$\Pr(B=q \mid C=f) = 2/5$$

Now we have a test example:

$$A = m \quad B = q \quad C = ?$$

An Example (cont ...)

- For class $C = t$, we have

$$\Pr(C = t) \prod_{j=1}^2 \Pr(A_j = a_j | C = t) = \frac{1}{2} \times \frac{2}{5} \times \frac{2}{5} = \frac{2}{25}$$

- For class $C = f$, we have

$$\Pr(C = f) \prod_{j=1}^2 \Pr(A_j = a_j | C = f) = \frac{1}{2} \times \frac{1}{5} \times \frac{2}{5} = \frac{1}{25}$$

- $C = t$ is more probable, thus t is the final class

Additional Issues

- ▶ **Numeric attributes:** Naïve Bayesian learning assumes that all attributes are categorical. Numeric attributes need to be discretized.
- ▶ **Zero counts:** An particular attribute value never occurs together with a class in training set. We need **smoothing**.

$$\Pr(A_i = a_i | C = c_j) = \frac{n_{ij} + \lambda}{n_j + \lambda S_i}$$

$$\Pr(C = c_j) = \frac{n_j + \lambda}{N + \lambda K}$$

- ▶ **Missing values:** Ignored

On naïve Bayesian classifier

► Advantages:

- Easy to implement
- Very efficient
- Good results obtained in many applications

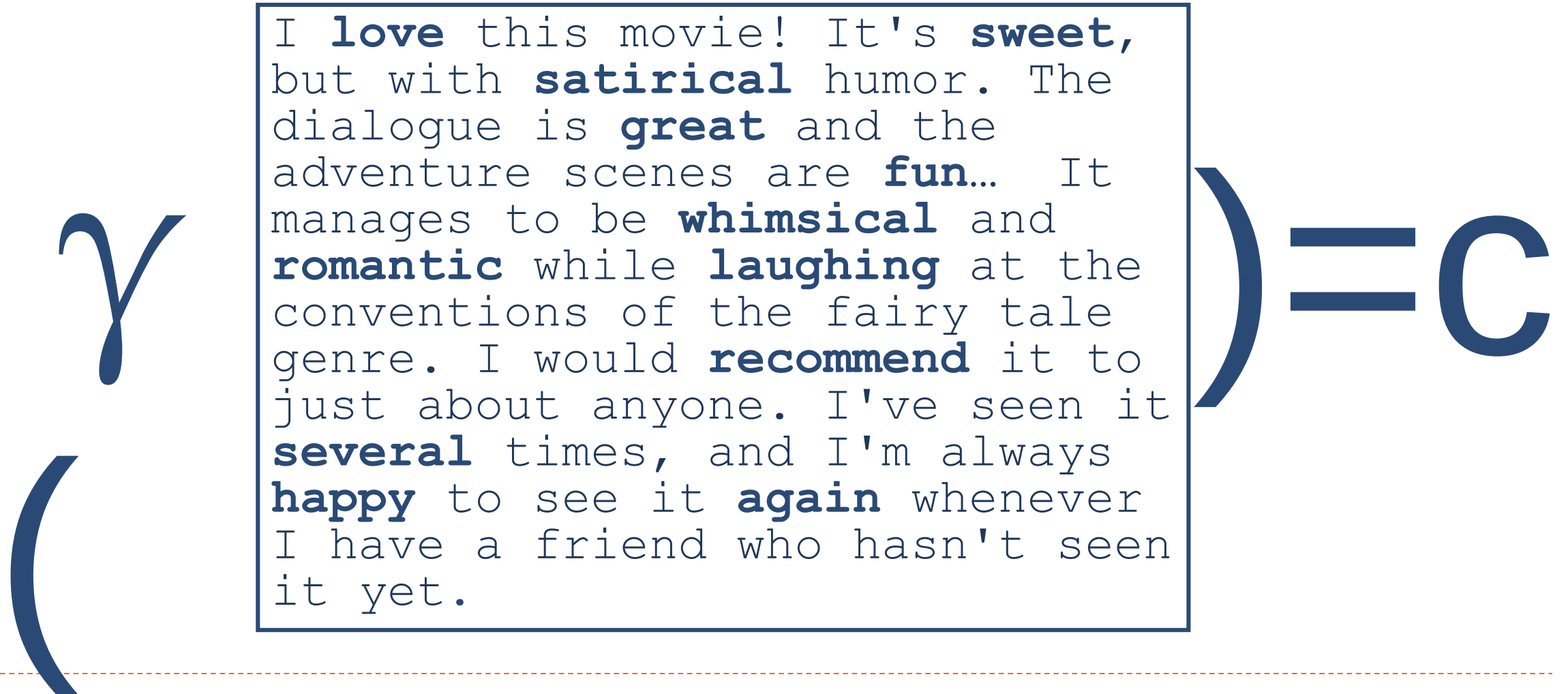
► Disadvantages

- Assumption: class conditional independence, therefore loss of accuracy when the assumption is seriously violated (those highly correlated data sets)

Naïve Bayes on Text Classification

$$\gamma \left(\begin{array}{l} \text{I love this movie! It's sweet,} \\ \text{but with satirical humor. The} \\ \text{dialogue is great and the} \\ \text{adventure scenes are fun... It} \\ \text{manages to be whimsical and} \\ \text{romantic while laughing at the} \\ \text{conventions of the fairy tale} \\ \text{genre. I would recommend it to} \\ \text{just about anyone. I've seen it} \\ \text{several times, and I'm always} \\ \text{happy to see it again whenever} \\ \text{I have a friend who hasn't seen} \\ \text{it yet.} \end{array} \right) = C$$

The bag of words representation



The bag of words representation: using a subset of words

$$\gamma \left(\begin{array}{l} \text{x love xxxxxxxxxxxxxxxxxxxx sweet} \\ \text{xxxxxxxx satirical xxxxxxxxxxxx} \\ \text{xxxxxxxxxxxxxxxx great xxxxxxxx} \\ \text{xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx fun xxxx} \\ \text{xxxxxxxxxxxxxxxxxxxxxxxx whimsical xxxx} \\ \text{romantic xxxx laughing} \\ \text{xx} \\ \text{xxxxxxxxxxxxxxxxxxxxxxxx recommend xxxxxx} \\ \text{xx} \\ \text{x several xxxxxxxxxxxxxxxxxxxxxxxxxxxx} \\ \text{xxxxx happy xxxxxxxxxxxx again} \\ \text{xx} \\ \text{xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx} \end{array} \right) = C$$

The bag of words representation

γ

great	2
love	2
recommend	1
laugh	1
happy	1
...	...

$= C$

$($

Bayes' Rule Applied to Documents and Classes

► For a document d and a class c

$$c_{MAP} = \operatorname{argmax}_{c \in C} P(c | d)$$

MAP is “maximum a posteriori” = most likely class

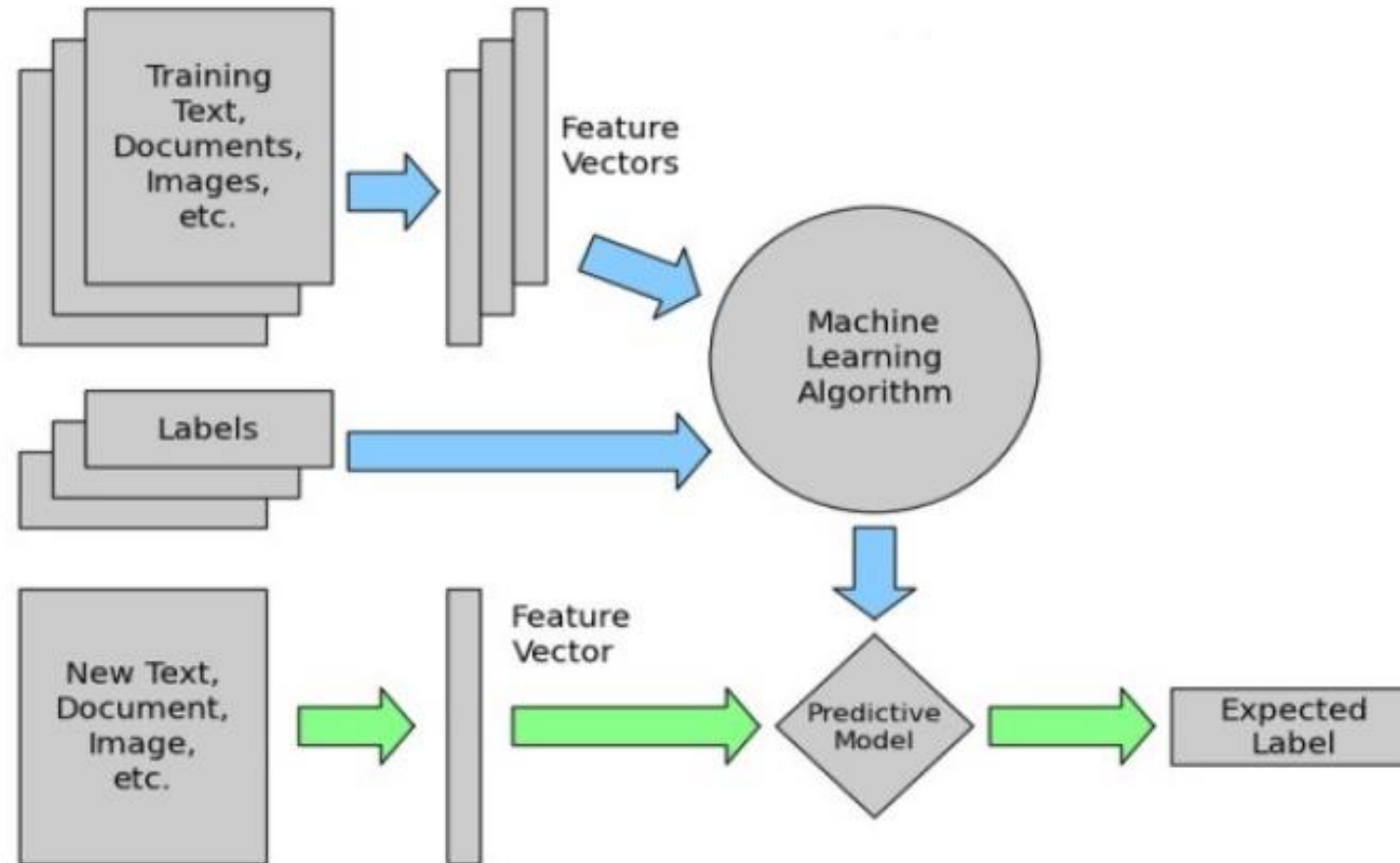
$$= \operatorname{argmax}_{c \in C} \frac{P(d | c)P(c)}{P(d)}$$

Bayes Rule

$$= \operatorname{argmax}_{c \in C} P(d | c)P(c)$$

Dropping the denominator

A General Workflow of Text Classification



A Simple Example

Feature vectors representing text contains the probabilities of appearance of the words of the text within the texts of a given category so that the algorithm can compute the likelihood of that text's belonging to the category.

Feature Vector

Document ID	Word 1	Word 2
1	2	0
2	0	1
3	3	1
4	1	3

ID	Class
1	1
2	0
3	0
4	1

1- belongs
0- Does not belong

A Simple Example

ID	Content	Class
1	Nvidia GPU is the best in the world.	computer graphics
2	Nvidia is giving tough competition to AMD.	computer graphics
3	We were running our application with GTX 1050(High end GPU) still it didn't work then we realized the problem was with the OS.	computer graphics
4	GPU, Ganpat Pandey University, is located in Maharashtra.	not computer graphics
5	Please buy GPU from our store.	?

A Simple Example

- ▶ We can predict the class of last data by using Naive Bayes by considering the probability of important words,
- ▶ Important words for com.graphics from the data can be considered as: {Nvidia, GPU, AMD, GTX 1050}
- ▶ All words will have feature vectors of theirs representing their number of occurrences in each record,
- ▶ Nvidia -> [1; 1; 0; 0] and so on.
- ▶ $P(\text{computer graphics} \mid \text{GPU}) = P(\text{GPU} \mid \text{computer graphics}) * (P(\text{computer graphics}) / P(\text{GPU}))$

A Simple Example

- ▶ $P(\text{computer graphics}) = \frac{\text{Number of records having class as computer graphics}}{\text{Number of total records}} = \frac{3}{4} = 0.75$
- ▶ $P(\text{GPU}) = \frac{\text{Number of records having GPU}}{\text{Total number of records}} = \frac{3}{4} = 0.75$
- ▶ $P(\text{GPU} \mid \text{computer graphics}) = \frac{\text{Number of records having computer graphics with GPU in it}}{\text{Total number of computer graphics' records}} = \frac{2}{3} = 0.66$
- ▶ $P(\text{computer graphics} \mid \text{GPU}) = 0.66 * (0.75/0.75) = 0.66$
- ▶ This is greater than 0.5 so we can predict that this text data will be belonging to computer graphics.
- ▶ Here, we have considered only one word GPU that is important in our test data but in real data we'll consider feature vectors of all words and then compute the probability for the class based on the occurrences of all the non primitive words.
- ▶ The main advantage is that you can get really good results when data available is not much and computational resources are scarce.



2.4 Evaluation of Classifiers

Evaluating classification methods

► Predictive accuracy:

$$\text{Accuracy} = \frac{\text{Number of correct classifications}}{\text{Total Number of test cases}}$$

► Efficiency

- time to construct the model
- time to use the model

Robustness: handling noise and missing values

Scalability: efficiency in disk-resident databases

Interpretability: understandable and insight provided by the model

Compactness of the model: size of the tree, or the number of rules.

Evaluation methods

- ▶ **Holdout set:** The available data set D is divided into two disjoint subsets,
 - ▶ the training set D_{train} (for learning a model)
 - ▶ the test set D_{test} (for testing the model)
- ▶ **Important:** training set should not be used in testing and the test set should not be used in learning.
 - ▶ Unseen test set provides a unbiased estimate of accuracy.
- ▶ **The test set is also called the holdout set. (the examples in the original data set D are all labeled with classes.)**
- ▶ **This method is mainly used when the data set D is large.**

Evaluation methods (cont...)

- ▶ **n-fold cross-validation:** The available data is partitioned into n equal-size disjoint subsets.
- ▶ Use each subset as the test set and combine the rest $n - 1$ subsets as the training set to learn a classifier.
- ▶ The procedure is run n times, which give n accuracies.
- ▶ The final estimated accuracy of learning is the average of the n accuracies.
- ▶ 10-fold and 5-fold cross-validations are commonly used.
- ▶ This method is used when the available data is not large.

Evaluation methods (cont...)

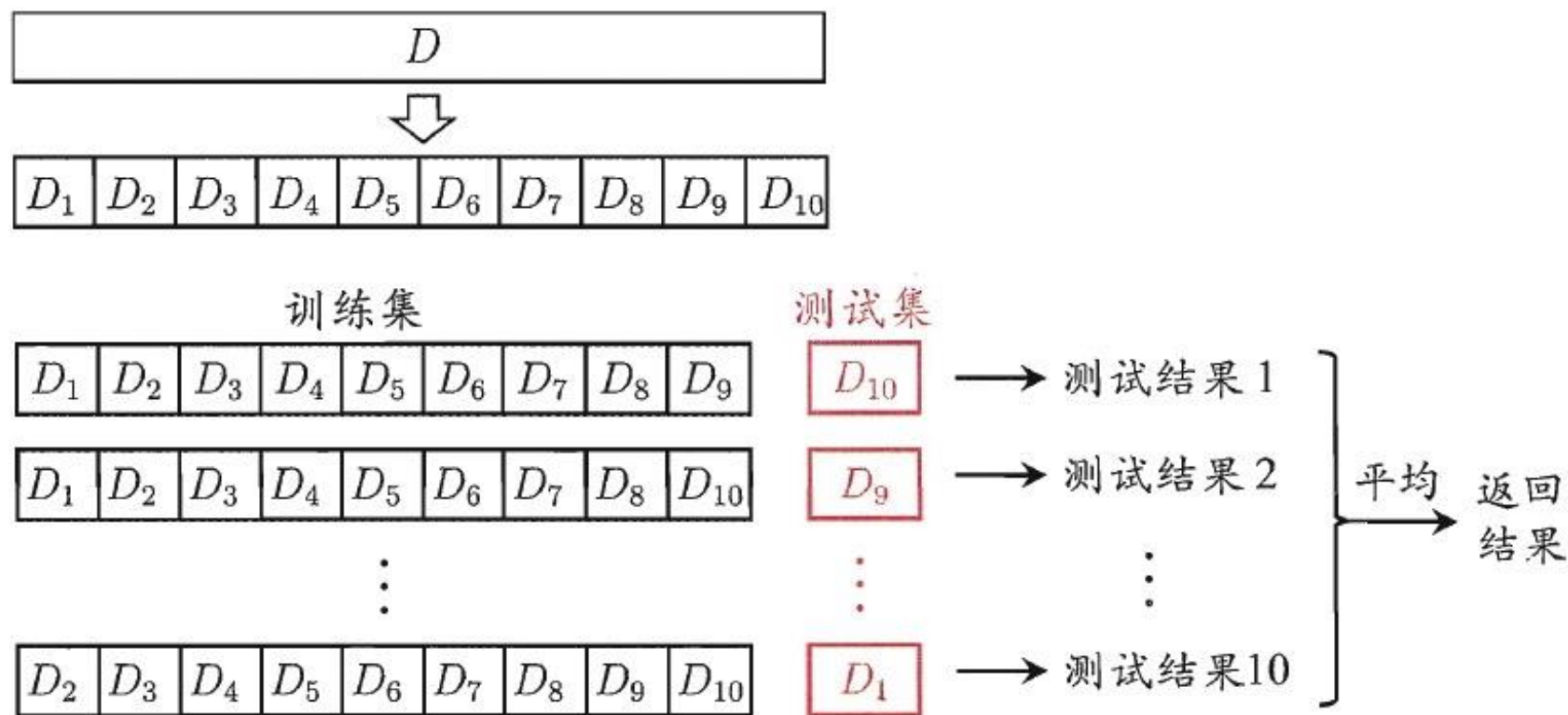


图 2.2 10 折交叉验证示意图

Evaluation methods (cont...)

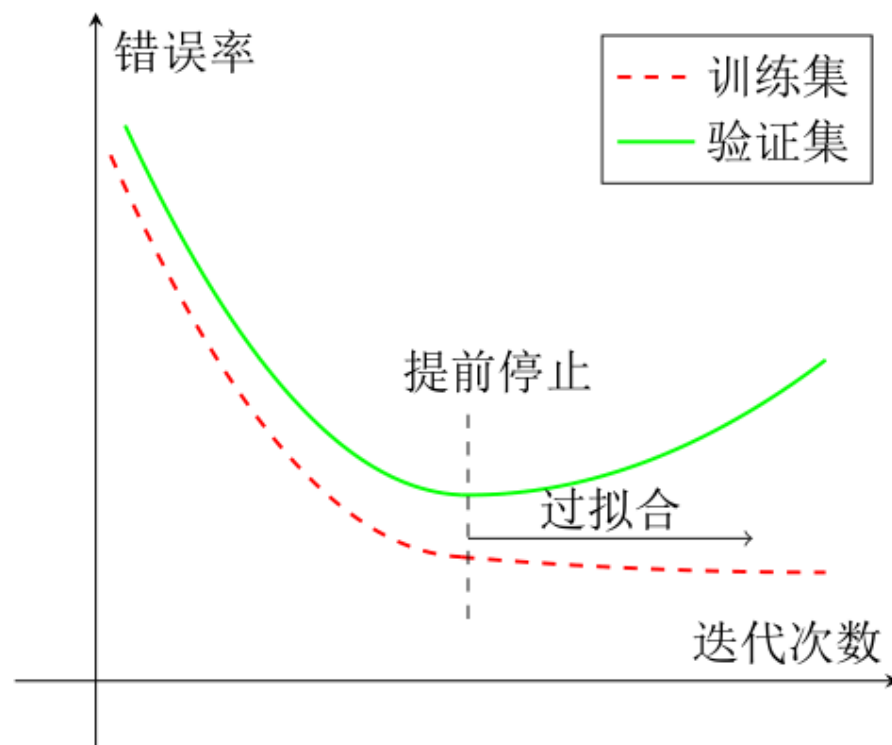
- ▶ **Leave-one-out cross-validation:** This method is used when the data set is very small.
- ▶ It is a special case of cross-validation
- ▶ Each fold of the cross validation has only a single test example and all the rest of the data is used in training.
- ▶ If the original data has m examples, this is m -fold cross-validation

Evaluation methods (cont...)

- ▶ **Validation set:** the available data is divided into three subsets,
 - ▶ A training set,
 - ▶ A validation set and
 - ▶ A test set.
- ▶ A validation set is used frequently for estimating parameters in learning algorithms.
- ▶ In such cases, the values that give the best accuracy on the validation set are used as the final parameter values.
- ▶ Cross-validation can be used for parameter estimating as well.

Early Stop

- Validation Dataset to test whether the parameters of each iteration are optimal on the validation dataset. If the error rate on the verification set no longer decreases, the iteration is stopped.



Classification measures

- ▶ Accuracy is only one measure (error = 1-accuracy).
- ▶ Accuracy is not suitable in some applications.
- ▶ In text mining, we may only be interested in the documents of a particular topic, which are only a small portion of a big document collection.
- ▶ In classification involving skewed or highly imbalanced data, e.g., network intrusion and financial fraud detections, we are interested only in the minority class.
 - ▶ High accuracy does not mean any intrusion is detected.
 - ▶ E.g., 1% intrusion. Achieve 99% accuracy by doing nothing.
- ▶ The class of interest is commonly called the positive class, and the rest negative classes.

Precision and recall measures

- ▶ Used in information retrieval and text classification.
- ▶ We use a confusion matrix to introduce them.

	Classified Positive	Classified Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

where

TP: the number of correct classifications of the positive examples (**true positive**),

FN: the number of incorrect classifications of positive examples (**false negative**),

FP: the number of incorrect classifications of negative examples (**false positive**), and

TN: the number of correct classifications of negative examples (**true negative**).

Precision and recall measures (cont...)

	Classified Positive	Classified Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

$$p = \frac{TP}{TP + FP} \quad r = \frac{TP}{TP + FN}$$

- **Precision** p is the number of correctly classified positive examples divided by the total number of examples that are classified as positive.
- **Recall** r is the number of correctly classified positive examples divided by the total number of actual positive examples in the test set.

An example

	Classified Positive	Classified Negative
Actual Positive	1	99
Actual Negative	0	1000

- ▶ **This confusion matrix gives**

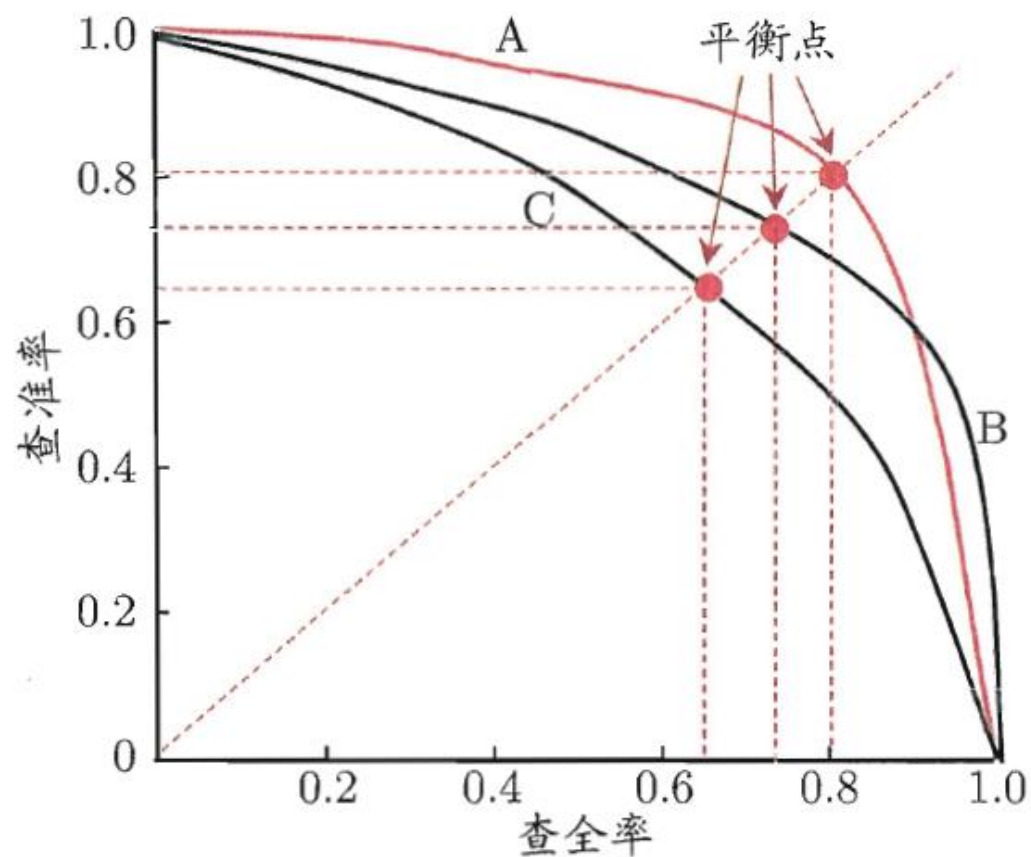
- ▶ precision $p = 100\%$ and

- ▶ recall $r = 1\%$

- because we only classified one positive example correctly and no negative examples wrongly.

- ▶ **Note: precision and recall only measure classification on the positive class.**

P-R Curve



1. According to the prediction results of the learner, the higher the ranking, the more likely it is to be a positive example.
2. If the samples are taken as positive examples one by one in this order, a set of precision and recall can be achieved.

Question?

Which one is better?

F1-value (also called F1-score)

- It is hard to compare two classifiers using two measures. F_1 score combines precision and recall into one measure

$$F_1 = \frac{2pr}{p+r}$$

$$H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

F_1 -score is the harmonic mean of precision and recall.

$$F_1 = \frac{2}{\frac{1}{p} + \frac{1}{r}}$$

- The harmonic mean of two numbers tends to be closer to the smaller of the two.
- For F_1 -value to be large, both p and r must be large.

Which one should we use, P or R?

- ▶ **Classification:** generate a real value or probability prediction for the test sample, and then compare this prediction value with a classification threshold. If it is greater than the threshold, it will be classified as positive, otherwise negative.
 - ▶ Equivalent to sorting, "most likely" is the positive case at the front, and "least likely" is the positive case at the back. The classification process is equivalent to dividing the sample into two parts by a certain "cut-off point" in this sorting, the former part is judged as positive cases, and the latter part is judged as negative cases.
 - ▶ If more attention is paid to P, the front position will be truncated, such as product recommendation.
 - ▶ If more attention is paid to R, the back position will be truncated, such as fugitive search.
-

Receive operating characteristics curve

- ▶ It is commonly called the ROC curve.
- ▶ It is a plot of the true positive rate (TPR) against the false positive rate (FPR).
- ▶ True positive rate:

$$TPR = \frac{TP}{TP + FN}$$

- ▶ False positive rate:

$$FPR = \frac{FP}{TN + FP}$$

Sensitivity and Specificity

- ▶ In statistics, there are two other evaluation measures:
 - ▶ Sensitivity: Same as TPR
 - ▶ Specificity: Also called True Negative Rate (TNR)

$$TNR = \frac{TN}{TN + FP}$$

- ▶ Then we have

$$FPR = 1 - specificity$$

Example ROC curves

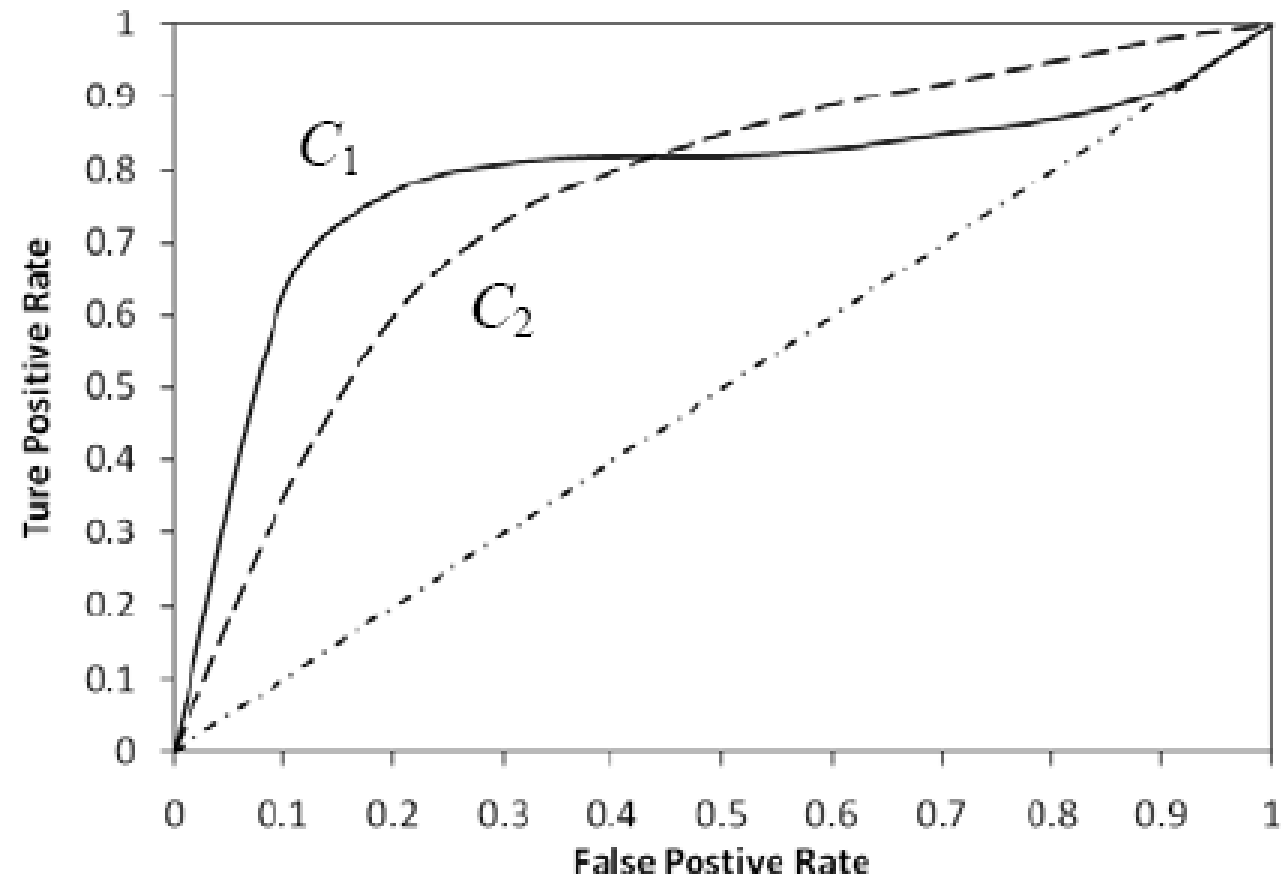
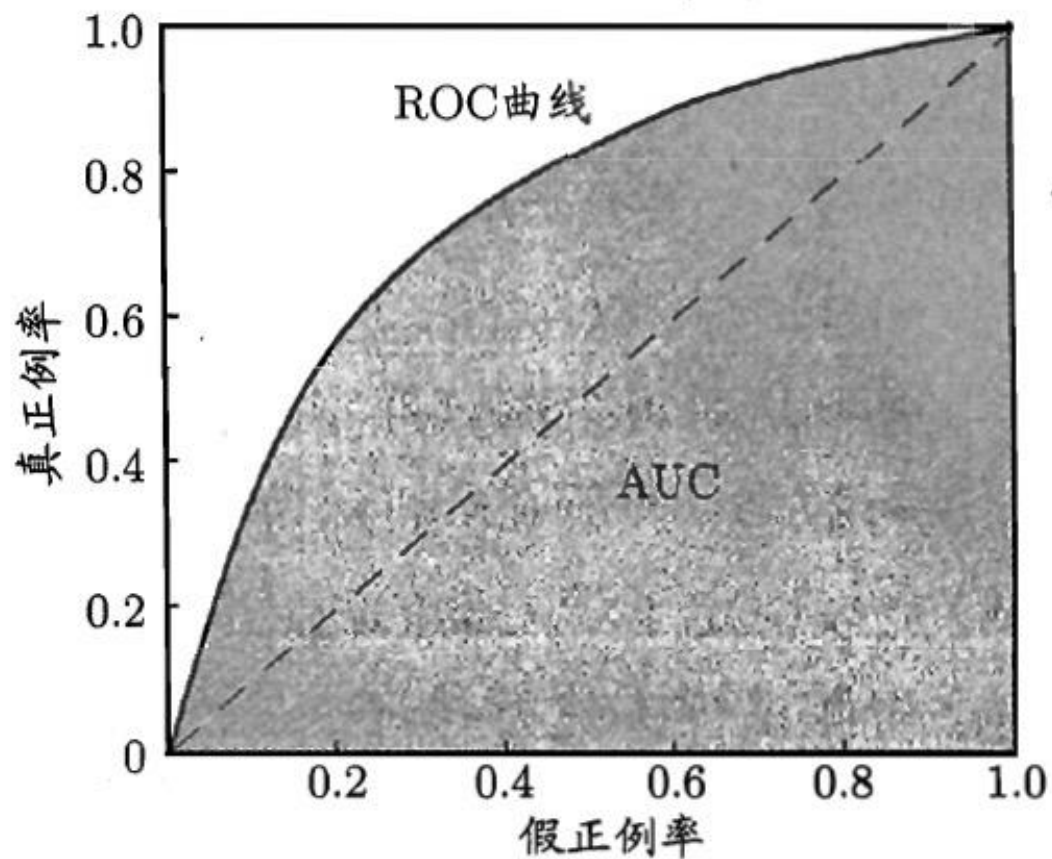


Fig. 3.8. ROC curves for two classifiers (C_1 and C_2) on the same data

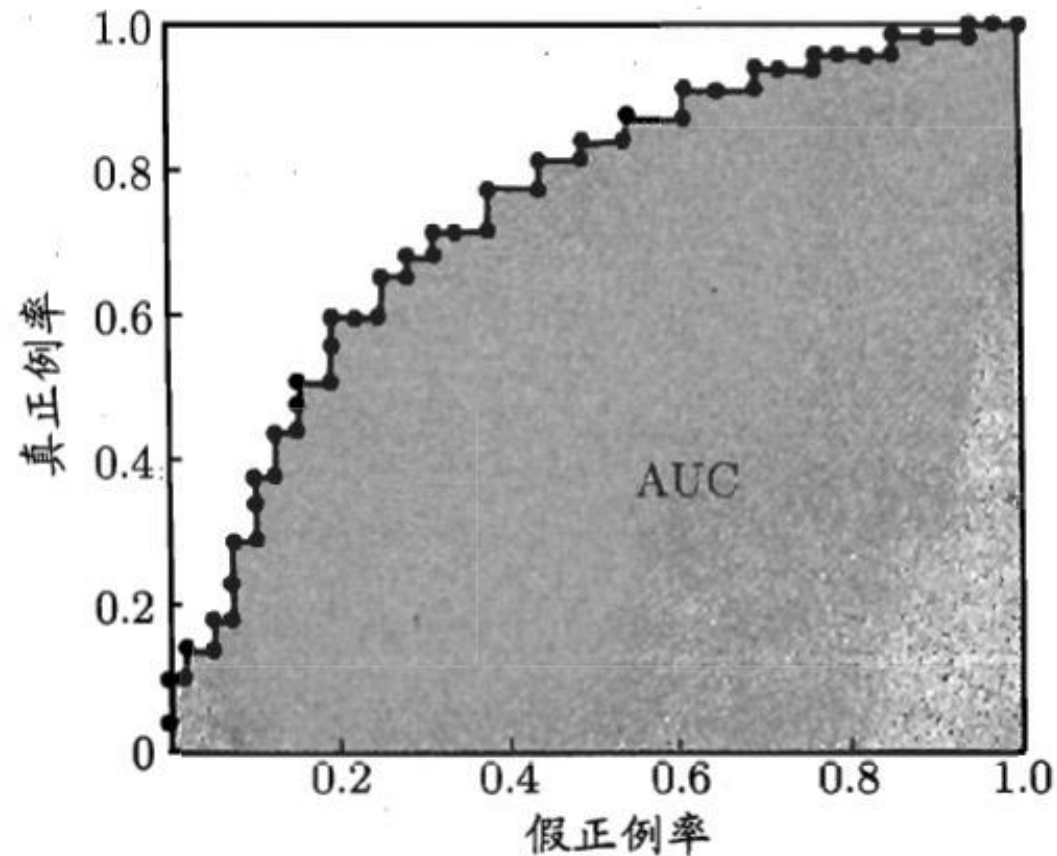
Area under the curve (AUC)

- ▶ Which classifier is better, C_1 or C_2 ?
 - ▶ It depends on which region you talk about.
- ▶ Can we have one measure?
 - ▶ Yes, we compute the area under the curve (AUC)
- ▶ If AUC for C_i is greater than that of C_j , it is said that C_i is better than C_j .
 - ▶ If a classifier is perfect, its AUC value is 1
 - ▶ If a classifier makes all random guesses, its AUC value is 0.5.

Drawing an ROC curve



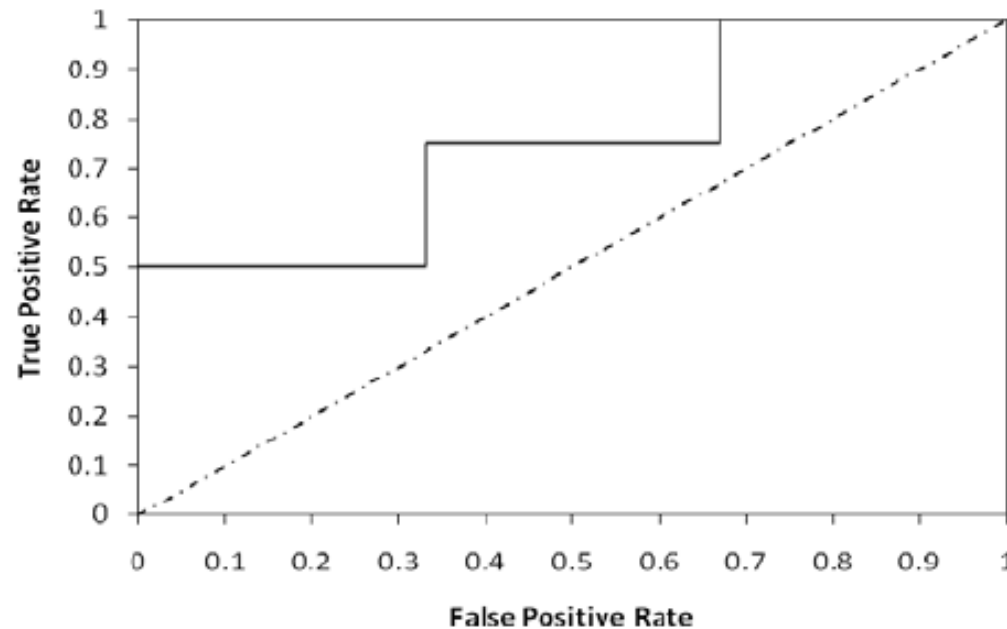
(a) ROC 曲线与 AUC



(b) 基于有限样例绘制的 ROC 曲线
与 AUC

Drawing an ROC curve

Rank		1	2	3	4	5	6	7	8	9	10
Actual class		+	+	-	-	+	-	-	+	-	-
TP	0	1	2	2	2	3	3	3	4	4	4
FP	0	0	0	1	2	2	3	4	4	5	6
TN	6	6	6	5	4	4	3	2	2	1	0
FN	4	3	2	2	2	1	1	1	0	0	0
TPR	0	0.25	0.5	0.5	0.5	0.75	0.75	0.75	1	1	1
FPR	0	0	0	0.17	0.33	0.33	0.50	0.67	0.67	0.83	1



Another evaluation method: Scoring and ranking

- ▶ Scoring is related to classification.
- ▶ We are interested in a single class (positive class), e.g., buyers class in a marketing database.
- ▶ Instead of assigning each test instance a definite class, scoring assigns a probability estimate (PE) to indicate the likelihood that the example belongs to the positive class.

Ranking and lift analysis

- ▶ After each example is given a PE score, we can rank all examples according to their PEs.
- ▶ We then divide the data into n (say 10) bins. A lift curve can be drawn according how many positive examples are in each bin. This is called lift analysis.
- ▶ Classification systems can be used for scoring. Need to produce a probability estimate.
 - ▶ E.g., in decision trees, we can use the confidence value at each leaf node as the score.

An example

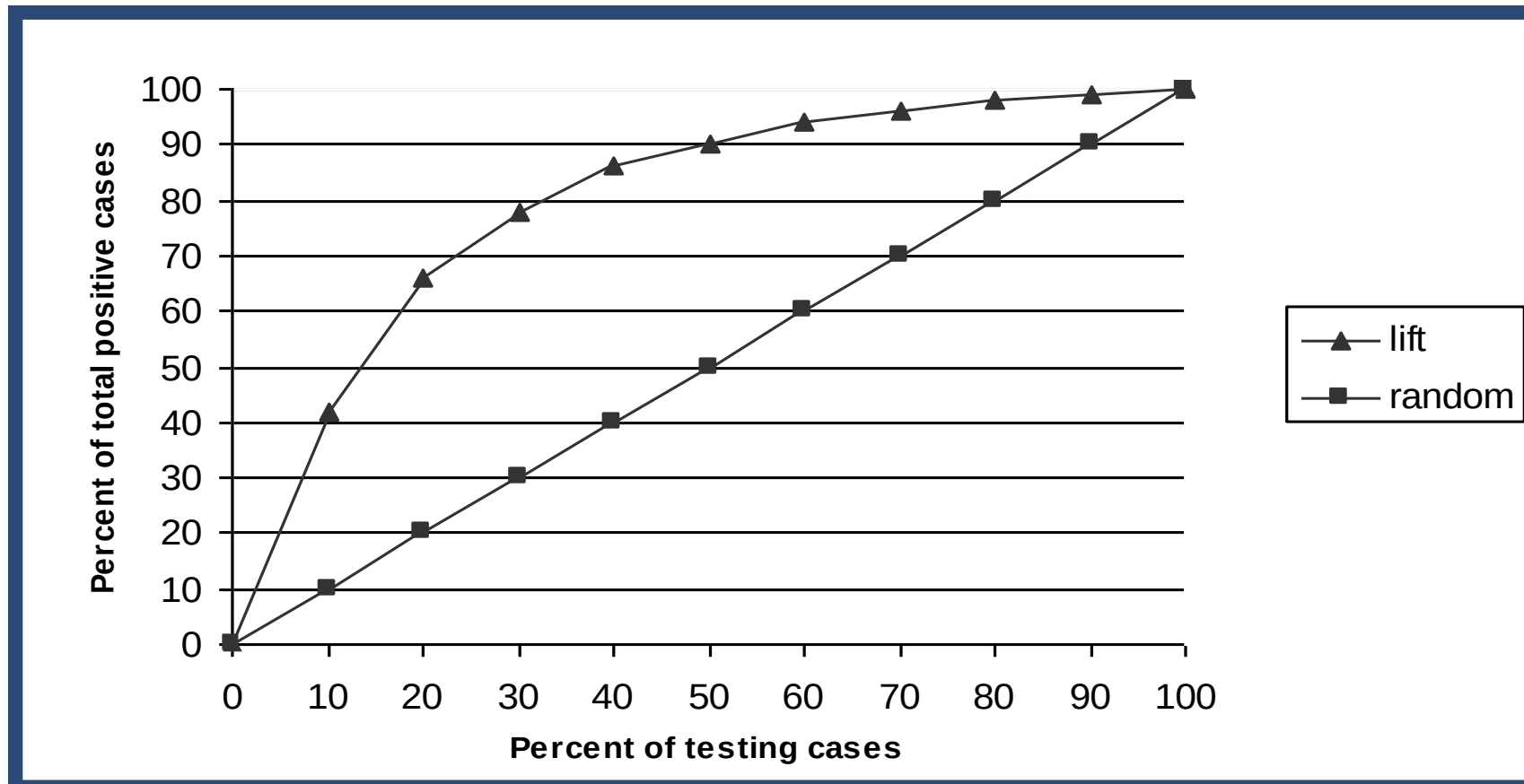
- ▶ We want to send promotion materials to potential customers to sell a watch.
- ▶ Each package cost \$0.50 to send (material and postage).
- ▶ If a watch is sold, we make \$5 profit.
- ▶ Suppose we have a large amount of past data for building a predictive/classification model. We also have a large list of potential customers.
- ▶ How many packages should we send and who should we send to?

An example

- ▶ Assume that the test set has 10000 instances. Out of this, 500 are positive cases.
- ▶ After the classifier is built, we score each test instance. We then rank the test set, and divide the ranked test set into 10 bins.
 - ▶ Each bin has 1000 test instances.
 - ▶ Bin 1 has 210 actual positive instances
 - ▶ Bin 2 has 120 actual positive instances
 - ▶ Bin 3 has 60 actual positive instances
 - ▶ ...
 - ▶ Bin 10 has 5 actual positive instances

Lift Curve

Bin	1	2	3	4	5	6	7	8	9	10
	210	120	60	40	22	18	12	7	6	5
	42%	24%	12%	8%	4.40%	3.60%	2.40%	1.40%	1.20%	1%
	42%	66%	78%	86%	90.40%	94%	96.40%	97.80%	99%	100%

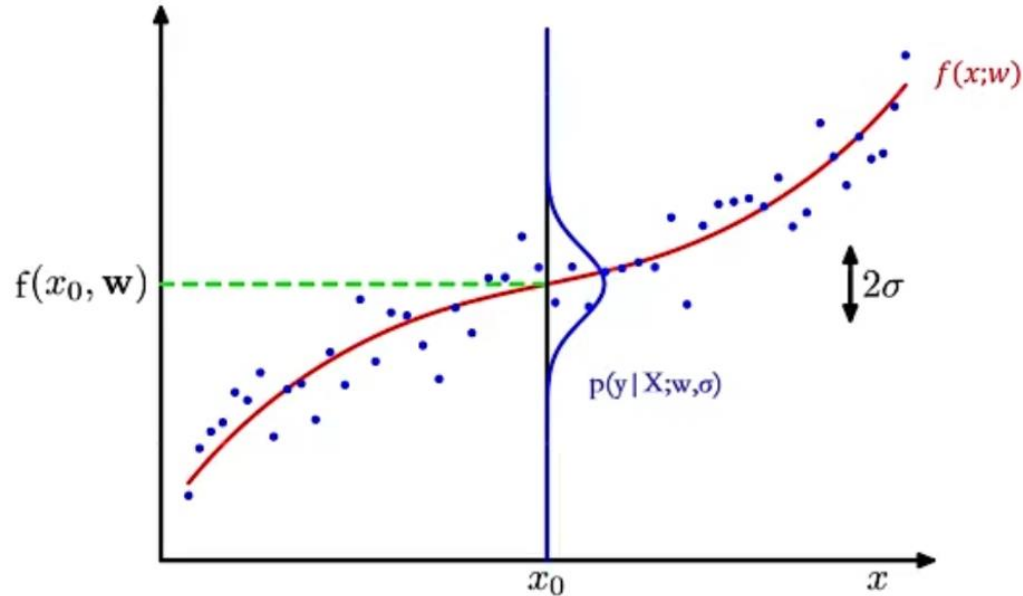




2.5 Linear Regression from a Probability Perspective

Linear regression from a probability perspective

- Assuming label y is a r.v., which follows the Gaussian distribution with the average $w^T x$ and variance σ^2



$$\begin{aligned} p(y|\mathbf{x}; \mathbf{w}, \sigma) &= \mathcal{N}(y; \mathbf{w}^\top \mathbf{x}, \sigma^2) \\ &= \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \mathbf{w}^\top \mathbf{x})^2}{2\sigma^2}\right) \end{aligned}$$

Likelihood Function (似然函数)

- ▶ The likelihood function is a function of the parameter w about the statistical model $p(x; w)$
- ▶ Probability $p(x; w)$ describes the distribution of the random variable x when the parameter w is fixed.
- ▶ Likelihood $p(x; w)$ describes the influence of different parameters w on the distribution of a known random variable x .

Maximum Likelihood Estimation (MLE)

► Assume $X_1, X_2, \dots, X_n$ are samples from X , the probability of the observed samples occurrence is $\prod_{i=1}^n p(x_i; w)$.

► Likelihood function:

$$L(w) = L(x_1, x_2, \dots, x_n; w) = \prod_{i=1}^n p(x_i; w)$$

► Maximize the likelihood function to get $\hat{w}$:

$$\hat{w} = \operatorname{argmax}_w L(x_1, x_2, \dots, x_n; w)$$

where $\hat{w}(x_1, x_2, \dots, x_n)$ is maximum likelihood estimation and $\hat{w}(X_1, X_2, \dots, X_n)$ is maximum likelihood statistic.

► Log Likelihood Equation:

$$dL(w)/dw = 0 \rightarrow d\log L(w)/dw = 0$$

Likelihood Function in Linear Regression

- Likelihood function of parameter w in training set D

$$p(\mathbf{y}|\mathbf{X}; \mathbf{w}, \sigma) = \prod_{n=1}^N p(y^{(n)}|\mathbf{x}^{(n)}; \mathbf{w}, \sigma) = \prod_{n=1}^N \mathcal{N}(y^{(n)}; \mathbf{w}^\top \mathbf{x}^{(n)}, \sigma^2)$$

- MLE finds a set of parameters w such that the likelihood function $p(\mathbf{y}|\mathbf{X}; w, \sigma)$

$$\frac{\partial \log p(\mathbf{y}|\mathbf{X}; \mathbf{w}, \sigma)}{\partial \mathbf{w}} = 0 \quad \longrightarrow \quad \mathbf{w}^{ML} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$$

Bayesian Perspective

▶ Maximum Likelihood:

- ▶ When the training data is relatively small, overfitting will occur, resulting in inaccurate parameter estimation
- ▶ Add prior (先验) knowledge to the parameters

▶ Bayesian Learning:

- ▶ Consider the parameter w as a random variable
- ▶ Objective: Given a set of observation data X , find the distribution $p(w|X)$ of parameter w
- ▶ $p(w|X)$ is also called posterior distribution (后验分布)

Basic Idea of Probability

► Bayesian rule

► The relationship between $p(y|x)$ and $p(x|y)$: $p(y|x) = \frac{p(x|y)p(y)}{p(x)}$

► Posterior, Likelihood, and prior

$$p(w|x) \propto p(x|w)p(w)$$

Posterior $\propto$ Likelihood $\times$ Prior

Bayesian Estimation

- ▶ Linear regression adopting Bayesian estimation is called Bayesian Linear Regression (贝叶斯线性回归)

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) = \frac{p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)}{\sum_{\mathbf{w}} p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)} \\ \propto \underbrace{p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma)}_{\text{likelihood}} p(\mathbf{w}; \nu),$$

后验
posterior

似然
likelihood

先验
prior

$$p(\mathbf{w}; \nu) = \mathcal{N}(\mathbf{w}; \mathbf{0}, \nu^2 \mathbf{I})$$

$$\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) \propto \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma) + \log p(\mathbf{w}; \nu)$$

$$\propto -\frac{1}{2\sigma^2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \mathbf{x}^{(n)} \right)^2 - \frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w},$$

$$= \underbrace{-\frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2}_{\text{data fit}} - \underbrace{\frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w}}_{\text{regularization}}$$

正则化系数 $\lambda = \sigma^2/\nu^2$

Maximum A Posteriori Estimation (MAP)

- ▶ Bayesian estimation is an interval estimation of parameters, that is, the distribution of parameters in an interval.
- ▶ If you want to get an optimal parameter value, you can use the maximum a posteriori estimation (MAP).

$$\mathbf{w}^{MAP} = \arg \max_{\mathbf{w}} p(\mathbf{y} | \mathbf{X}, \mathbf{w}; \sigma) p(\mathbf{w}; \nu)$$



$$\mathbf{w}^{MAP} = -\frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}^T \mathbf{w}\|^2 - \frac{1}{2\nu^2} \mathbf{w}^T \mathbf{w}$$

正则化系数 $\lambda = \sigma^2/\nu^2$

Conclusion

	Without prior	With prior
square error	Empirical risk minimization	Structural risk minimization
probability	MLE	MAP

$$\mathbf{w}^{ML} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y} \quad \mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$$



2.6 Bias-Variance Decomposition

How to select a suitable model?

▶ Model Selection

- ▶ Models with strong fitting ability are generally more complex and easy to overfitting.
- ▶ If the complexity of the model is limited and the fitting ability is reduced, it may be underfitted.

▶ How to select a model?

- ▶ The more complex the model, the lower the training error.
- ▶ Model cannot be selected according to the lowest training error.
- ▶ When selecting a model, the test set is not visible.

Bias-Variance Decomposition

► Expected risk:

$$\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim p_r(x,y)} \left[(y - f(x))^2 \right]$$

► The best model that ML algorithm can learn:

$$f^*(x) = \mathbb{E}_{y \sim p_r(y|x)} [y]$$

► Expected risk can be decomposed into

$$\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim p_r(x,y)} \left[(y - f^*(x) + f^*(x) - f(x))^2 \right]$$

$$= \mathbb{E}_{x \sim p_r(x)} \left[(f(x) - f^*(x))^2 \right] + \epsilon$$

$$\epsilon = \mathbb{E}_{(x,y) \sim p_r(x,y)} \left[(y - f^*(x))^2 \right]$$

通常是由于样本分布以及噪声引起的，无法通过优化模型来减少。

真实目标：当前模型与最优模型之间的差距

Bias-Variance Decomposition

- ▶ **Actually training a model, different training sets get different models.**
 - ▶ The ability of a ML algorithm (including model and optimization algorithm) can be evaluated by the average performance of models on different training sets.
 - ▶ $f_{\mathcal{D}}(x)$ is the model learned from training set $\mathcal{D}$.
 - ▶ For a single samples:

$$\begin{aligned} & \mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}) \right)^2 \right] \\ &= \mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] + \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2 \right] \\ &= \underbrace{\left(\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2}_{(\text{bias.x})^2} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(\mathbf{x})] \right)^2 \right]}_{\text{variance.x}} \end{aligned}$$

Bias-Variance Decomposition

► Expected error

$$\begin{aligned}\mathcal{R}(f) &= \mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\mathbb{E}_{\mathcal{D}} \left[(f_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2 \right] \right] + \epsilon, \\ &= (\text{bias})^2 + \text{variance} + \epsilon.\end{aligned}$$

► Expected error is decomposed into

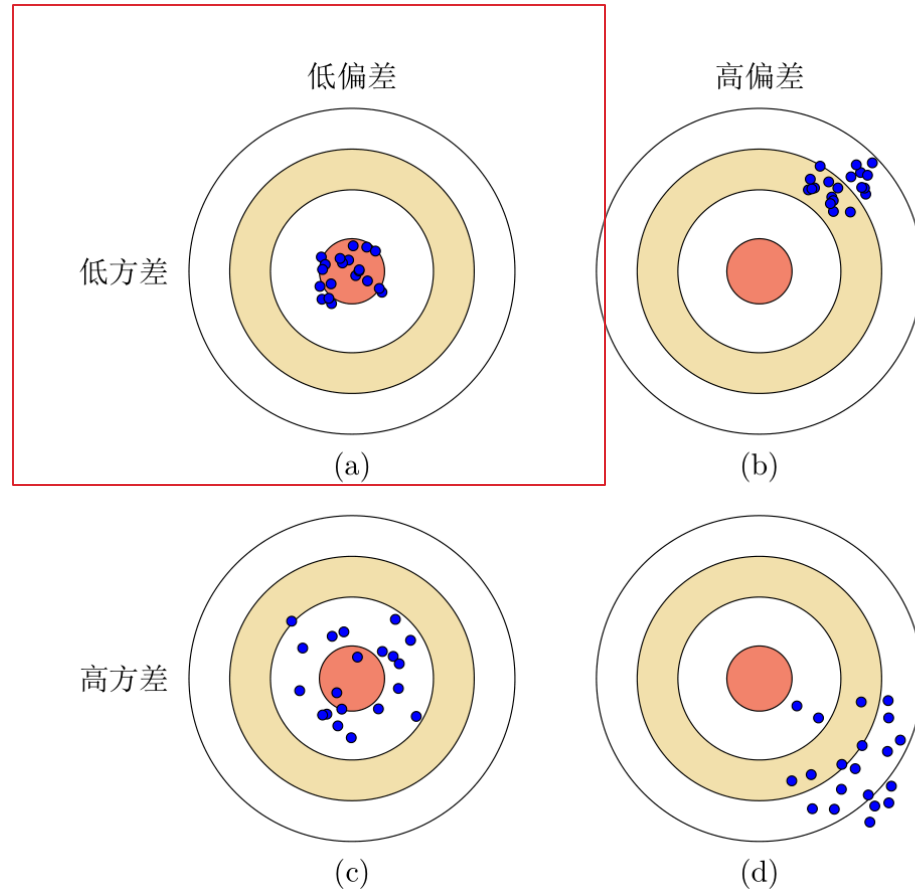
$$\mathcal{R}(f) = (\text{bias})^2 + \text{variance} + \epsilon.$$

$$\mathbb{E}_{\mathbf{x}} \left[\left(\mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2 \right]$$

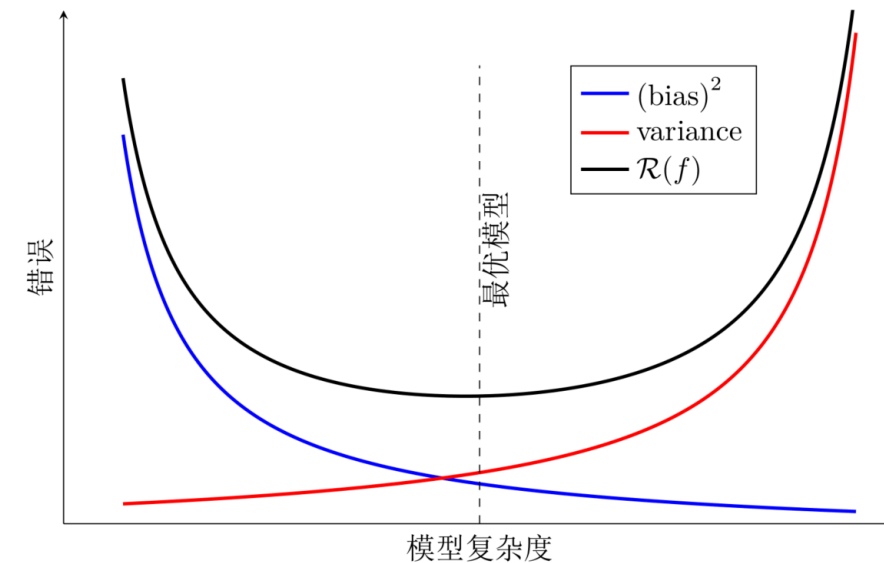
$$\mathbb{E}_{(\mathbf{x}, y) \sim p_r(\mathbf{x}, y)} \left[(y - f^*(\mathbf{x}))^2 \right]$$

$$\mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{\mathcal{D}} \left[(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})])^2 \right] \right]$$

Model Selection: Deviation and Variance



Minimizing expected risk is equivalent to minimizing the sum of deviation and variance.



Ensemble model: an effective method to reduce variance

Common Theorems

► No Free Lunch Theorem (没有免费午餐定理)

- For the iterative optimization algorithm, there is no algorithm that is effective for all problems (in a limited search space). If an algorithm is effective for some problems, it must be worse than the pure random search algorithm for others.



Common Theorems

- ▶ **Ugly Duckling Theorem (丑小鸭定理)**
 - ▶ The difference between an ugly duckling and a white swan is as great as that between two white swans.



Common Theorems

- ▶ **Occam's Razor (奥卡姆剃刀原理)**
 - ▶ If it is not necessary, do not add entities.



Inductive Bias (归纳偏置)

- ▶ Many learning algorithms often make some assumptions about learning problems, which are called inductive bias.
- ▶ In the nearest neighbor (KNN) classifier, we will assume that most samples in a small local area belong to the same category in the feature space.
- ▶ In Naïve Bayes classifier, we will assume that the conditional probability of each feature is independent of each other.
- ▶ Inductive bias is often called Prior in Bayesian learning.

PAC Learning

- ▶ PAC: Probably Approximately Correct
- ▶ According to the law of large numbers, when the training set size $|D|$ tends to infinity, the generalization error tends to zero, the empirical risk tends to the expected risk.

▶ PAC Learning

$$\lim_{|D| \rightarrow \infty} \mathcal{R}(f) - \mathcal{R}_D^{emp}(f) = 0$$

$$P\left(\underbrace{(\mathcal{R}(f) - \mathcal{R}_D^{emp}(f)) \leq \epsilon}_{\text{Approximate correctness, } 0 < \epsilon < 0.5}\right) \geq 1 - \delta$$

probably, $0 < \delta < 0.5$

Sample Complexity

- ▶ For a fixed ϵ, δ , The sample complexity can be calculated in turn as follows

$$n(\epsilon, \delta) \geq \frac{1}{2\epsilon^2} (\ln |\mathcal{F}| + \ln \frac{2}{\delta})$$

- ▶ Where $|\mathcal{F}|$ is the size of the hypothetical space, which can be measured by Rademacher complexity or VC dimension.
- ▶ PAC learning theory can help to analyze under what conditions a ML method can learn an approximately correct classifier.
- ▶ If we hope that the larger the hypothesis space of the model is, the smaller the generalization error is, and the more samples it needs.